Visualization for Quantum Computing: Current State and Future Directions

Feng Liang, Xiaolin Wen, Shixian Zhou, Shaolun Ruan, Yuxuan Du, Qiang Guan, Jun Ye, and Yong Wang

***Abstract*—Visualization has played an essential role in understanding quantum computing (QC), from the Bloch Sphere to interactive visual analytics systems. Yet existing surveys either treat visualization as part of broader interface, education, or computer-graphics discussions, or focus on quantum rendering and data-processing pipelines; none provides a comprehensive visualization-centered taxonomy for this emerging area of visualization for quantum computing (VIS4QC). We address this gap by combining a venue search from 2015 to 2025 with agentic search and snowballing, curating 69 core publications and tools, and developing a three-dimensional taxonomy informed by QC expert co-authors. The taxonomy characterizes VIS4QC by *why* visualizations are created (Learning & Interpretation vs. Productivity & Utility), *what* QC subjects they address (Qubits & States, Gates & Circuits, Machines & Systems, and Algorithm Development), and *how* they visualize QC data through visual representation types and data-specific strategies. The corpus remains small but has grown rapidly since 2023, suggesting that VIS4QC is still emerging. Our support-coverage analysis shows a disconnect between academia and industry: academic work mainly supports learning about quantum states, while industry tools mainly support productivity around circuit diagrams. Visualization research provides limited tooling for practitioners, while industry tools incorporate few of the richer visual representations developed in academia. Algorithm Development and Machines & Systems receive the least coverage, and productivity tools for Algorithm Development are especially sparse despite the importance of algorithm development for practical QC applications. These gaps surface in quantum education and quantum software engineering, where users need connected support across concepts, artifacts, workflows, and outcomes. Building on this analysis, we identify future directions across quantum computing (QC) subject categories: scalable visualization of quantum states, visual support for dynamic and transpiled circuits, visual analytics for quantum hardware statistics, and mechanism-based visualization of quantum algorithms. We provide an online gallery for exploring the curated corpus and taxonomy: https://vis4qc.vercel.app/.**

***Index Terms*—Visualization, quantum computing, quantum state, quantum circuit, visual analytics, survey.**

I. Introduction

QUANTUM computing (QC) promises computational capabilities beyond classical computing. Quantum algorithms can theoretically solve many classically intractable problems with considerable speedup. For example, Shor's Algorithm [1], a cornerstone of QC research, is exponentially faster than known classical alternatives in factoring integers. Since then, numerous algorithms have been developed to exploit the computational power of quantum systems [2]–[5]. On the hardware side, current quantum computers remain noisy but are increasingly used in practical application studies (e.g., [6], [7]), alongside selected demonstrations reporting quantum utility or beyond-classical performance [8], [9].

These developments have moved QC toward broader experimentation and use. The quantum industry has therefore developed various tools and software development kits (SDKs) (e.g., Qiskit [10], PennyLane [11], Braket [12], and CUDA-Q [13]) to make QC more accessible. Most include built-in visualization features that help users (e.g., physicists, students, developers, and engineers) understand various aspects of QC. In academia, visualization has also been essential for understanding QC and the underlying quantum physics. For example, the Bloch Sphere [14], invented decades before Feynman's proposal of quantum computation [15], is the earliest visual representation of quantum states and remains fundamental for teaching and understanding QC today. Many more visual representations, tools, and systems have since been developed for different use cases.

As QC continues to evolve, its growing user base will lead to increasing needs for visualizations for understanding, debugging, and optimizing QC systems. Despite this importance, existing surveys either treat visualization as part of broader interface, education, or computer-graphics discussions, or focus on quantum rendering and data-processing pipelines. To the best of our knowledge, no existing survey provides a comprehensive visualization-centered taxonomy of Visualization for Quantum Computing (VIS4QC) that maps the research landscape and guides future work. Because VIS4QC lies at the intersection of visualization and QC, a clear taxonomy is essential for researchers to navigate both fields, which, however, still does not exist. Without these, newcomers lack a structured entry point into the field, and researchers cannot systematically identify which needs remain underexplored.

A useful VIS4QC taxonomy must address three fundamental questions. 1) *why are visualizations created?* Purposes, such as teaching newcomers versus supporting expert workflows, lead to distinct design requirements. 2) *what QC subjects are visualized?* Quantum computing spans subjects from individual qubits to full algorithms, each with distinct data types and visualization needs; yet, existing work focuses on only a few. 3) *how are QC subjects visualized?* Existing techniques face different challenges across these subjects, from

Feng Liang, Xiaolin Wen, Yuxuan Du, and Yong Wang are with Nanyang Technological University. E-mail: {feng011, xiaolin004}@e.ntu.edu.sg, {yuxuan.du, yong-wang}@ntu.edu.sg. Y. Wang is the corresponding author.

Shixian Zhou is with Hangzhou Dianzi University. E-mail: 241330022@hdu.edu.cn.

Shaolun Ruan is with University of Konstanz. E-mail: shaolun.ruan@uni-konstanz.de.

Qiang Guan is with Kent State University. E-mail: qguan@kent.edu.

Jun Ye is with CETC International Cornerstone Quantum Industry (Suzhou) Co., Ltd; E-mail: yjmaxpayne@hotmail.com.

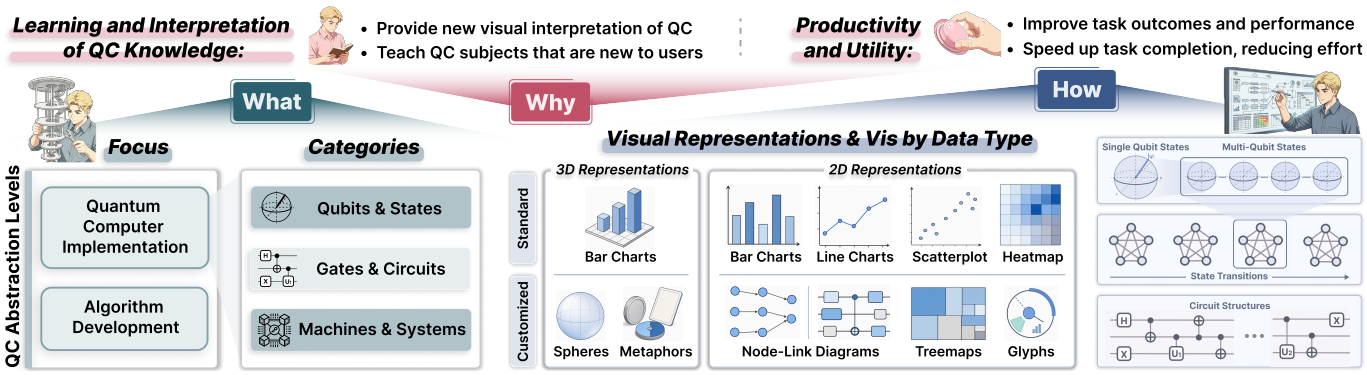


Fig. 1. Overview of our three-dimensional taxonomy for VIS4QC. Why (top) distinguishes visualizations created for *Learning & Interpretation* from those for *Productivity & Utility* in Section IV. What (left) organizes QC subjects by abstraction level, separating *Quantum Computer Implementation* from *Algorithm Development*, with implementation-focused subjects further categorized into *Qubits & States*, *Gates & Circuits*, and *Machines & Systems* in Section V. How (right) summarizes visualizations through two linked views: visual representations, organized by dimensionality and customization, and data-specific strategies for *computational data*, including *single-qubit states*, *multi-qubit states*, *state transitions*, and *circuit structures* in Section VI.

scalability limits for quantum states to the lack of visualization support for full algorithms. Fig. 1 overviews the taxonomy.

To provide both a comprehensive survey and a systematic taxonomy that answers these questions, we collected and analyzed existing VIS4QC publications and tools (PTs). We performed an extensive venue search of academic journals and conferences spanning QC, Visualization (VIS), Computer Graphics (CG), Human-Computer Interaction (HCI), and Software Engineering (SE), covering 41666 publications from 2015 to 2025, as shown in Table B.1 in Appendix. We also used agentic search tools (e.g., Perplexity Deep Research) to discover relevant PTs from the broader Internet. Additionally, we performed recursive snowballing to ensure that earlier work was not missed. After filtering out irrelevant PTs, we compiled 69 core PTs (Table 1). We then manually analyzed them, distilled a taxonomy, and coded them accordingly.

Our taxonomy characterizes the research space of VIS4QC along three dimensions: Usage & Purpose (UP), QC Subjects (QCS), and Visualizations, which correspond to the three fundamental questions above, respectively. The UP dimension captures **why** a visualization is created, either for *Learning & Interpretation (LI)*, where visualizations help users understand QC concepts, or for *Productivity & Utility (PU)*, where visualizations help experienced users perform QC tasks more efficiently. The QCS dimension captures **what** is visualized, organized by abstraction level from *Qubits & States (QS)*, through *Gates & Circuits (GC)*, to *Machines & Systems (MS)*, as well as *Algorithm Development (AD)* at a higher conceptual level. The Visualizations dimension captures **how** existing work turns QC data into visual form through two linked views. It first characterizes visual representation types, including spheres, charts, node-link diagrams, treemaps, and glyphs. It then analyzes how these representations are used for computational data, such as quantum states and circuit structures, and for operational data, such as hardware and execution metrics.

Despite this broad, multi-stage search, the resulting corpus of 69 core PTs remains relatively small. Its rapid growth after 2023 points to rising interest, but the limited corpus suggests that VIS4QC is still an emerging area, with substantial room for visualization research beyond the support needs and QC subjects studied so far. Examining support coverage across the UP and QCS dimensions, together with the source of PTs, reveals different emphases in academia and industry. Academia concentrates on visualizing quantum states for learning (33 out of 35 related PTs are from academia), while industry provides circuit diagrams for productivity (17 out of 30 related PTs are industry tools). The surveyed work shows a gap between visualization research and practice: research emphasizes conceptual explanation, while many industry tools rely on conventional circuit and state views. *AD* and *MS* receive the least coverage overall, and productivity tools for *AD* are nearly absent (4 PTs) despite algorithm development being central to realizing practical QC applications. With real-world quantum applications (e.g., finance [6] and drug discovery [7]) emerging, these underexplored regions represent significant opportunities for visualization researchers. We further identify visualization challenges that differ across QC subject categories: scaling quantum-state visualizations, representing dynamic and transpiled circuits, analyzing quantum hardware statistics, and visualizing algorithmic mechanisms rather than state evolution. Together, these support gaps call for broader VIS4QC research, especially on underexplored areas such as *AD* and *MS*.

In summary, this survey makes the following contributions:

- A comprehensive collection of 69 core PTs in VIS4QC (see Table 1), providing visualization researchers with an entry point into the field.
- A three-dimensional taxonomy that defines the research space of VIS4QC along *why* (Usage & Purpose), *what* (QC Subjects), and *how* (visual representation types and data-specific strategies), enabling systematic navigation of this interdisciplinary space.
- A support-coverage analysis that maps visualization support and gaps across Usage & Purpose and QC Subjects, explains how these gaps arise in quantum education and quantum software engineering, and derives future directions across QC subject categories.

This survey is organized as follows. Section II reviews and compares related surveys. Section III details our PT collection and taxonomy construction. Section IV, V, and VI present our three-dimensional taxonomy: *why* (Usage & Purpose), *what*

(QC Subjects), and *how* (visual representation types and data-specific strategies), respectively. Section VII discusses current VIS4QC support coverage across UP and QCS, the gaps in quantum education and quantum software engineering and capabilities and limitations of current designs. Section VIII derives future directions across QC subject categories based on the analysis in Section VII. Section IX concludes the survey. Appendix A provides a QC tutorial relevant to visualization research. Appendix B provides additional methodology details. Appendix C provides selected visualization examples. Appendix D provides a list of all abbreviations used. Each reference to an item in Appendix is prefixed with letter "A", "B", or "C" (e.g., Table B.1).

II. Related Surveys

Quantum Human Computer Interaction (QHCI), education and games are relevant to VIS4QC. QHCI studies address the interaction design between users and quantum computers. Although VIS4QC intersects with QHCI, the two differ in scope: QHCI treats visualization as one component of broader interface design, whereas VIS4QC focuses on visualization itself as the primary research contribution.

Quantum Human Computer Interaction: Ashktorab et al. identified four main topics of QHCI from their expert interviews [16]. They also surveyed existing tools and educational resources. However, as QC has since progressed rapidly, this survey no longer captures the current landscape of QHCI and VIS4QC research. More recently, Kim et al. [17] surveyed tools for QC, discussed challenges in QC interfaces, and proposed design principles with prototypes. As it focused on QHCI, it did not discuss VIS4QC research in depth but treated visualization as an engineering feature of quantum tools. Its taxonomy focused on tool utility, so it did not map the QC landscape from a visualization perspective or identify research directions at the intersection of the two fields.

VIS4QC: A survey by Bethel et al. [18] discussed VIS4QC alongside quantum rendering algorithms. Regarding VIS4QC, it summarized several representations of quantum states and visualizations for quantum circuits and algorithms. Despite its visualization framing, its primary focus was on quantum rendering and computer graphics rather than visualization research, and it did not examine VIS4QC research in depth or provide a taxonomy. More recently, Bethel et al. [19] examined research challenges and opportunities in using QC to accelerate visualization workloads. They briefly discussed visualizations of quantum states and circuits, identifying scalability and entanglement as challenges in VIS4QC. Their discussion outlined broad research directions, but a framework is still needed to organize existing VIS4QC techniques and analyze their coverage across usage purposes and QC subjects.

Quantum Education and Quantum Games: A review of quantum education is beyond our scope, but education research motivates the need for better visual support. Doyle et al. [20] highlight challenges in teaching quantum concepts to interdisciplinary learners and call for approaches that balance mathematical rigor and conceptual accessibility. Lane et al. [21] summarize broader challenges and future directions in quantum education, including the role of visualization. Quantum games are also relevant: surveys and a book chapter [22]–[24] summarize games and storytelling for quantum physics and QC. However, they focus on educational, entertainment, and game-design aspects rather than visualization research.

In contrast, our work focuses on VIS4QC, provides a comprehensive survey of existing studies, a concise background on relevant QC concepts, and a taxonomy that captures the current research landscape and informs future research directions.

III. Methodology

To map the research space of VIS4QC, we analyzed 69 core PTs closely related to VIS4QC in QC, VIS, CG, HCI, and SE. These PTs are coded based on the UP of visualizations (Section IV) and the visualized QC subjects (Section V). We developed the Visualizations dimension, *how* QC subjects are visualized (Section VI), through an analytical review of the visualizations of the core PTs. We used Large Language Model (LLM) agents for data collection and summarization, with manual verification at each step, as detailed in the following section. All analysis and writing are human-authored.

A. Survey Scope

We began our analysis by collecting a database of potentially relevant PTs. To achieve extensive coverage, we used three complementary methods in our collection pipeline, illustrated in Fig. B.5. Venue search systematically covers major publication venues across the fields related to VIS4QC. Agentic search extends the collection to online tools and relevant work outside the selected venues because earlier studies appear across scattered venues in physics, education, and theoretical computer science (e.g., [25]–[30]). Recursive snowballing further expands the collection from the references, related-work discussions, and documentation of collected PTs.

Venue Search: We selected the main journals and conferences in QC, VIS, CG, HCI, and SE based on their relevance to VIS4QC. The selected venues and their statistics are shown in Table B.1. We collected publications from the venues from 2015 to 2025 using official tools. We also collected QC tools referenced by these publications. The time range was chosen because: (1) this period covers the majority of relevant VIS4QC research; (2) as QC research is evolving rapidly, recent studies are likely to be more relevant; and (3) we cover relevant research prior to 2015 through agentic search and snowballing. The full venue collection contains 41666 publications. Due to the large volume, we used keyword screening on titles, abstracts, and full text. For VIS, HCI, and SE venues, we searched for "quantum"; for QC venues, we searched for "visual*", "interface", and "interact*", where "*" is a wildcard character. This broad screening preserved potentially relevant work for later manual filtering.

Agentic Search: We used multiple web-based agentic search tools to discover relevant PTs outside the selected venues. We manually checked each candidate in the results. The details are provided in Appendix B.

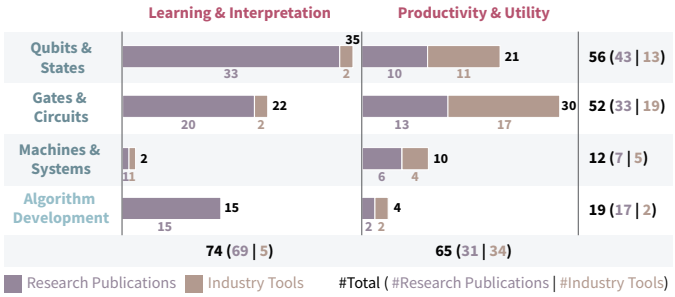


Fig. 2. Support coverage of all PTs across Usage & Purpose (columns) and QC Subjects (rows). Each stacked bar separates research publications from industry tools. The numbers count *assignments* of PTs to cells.

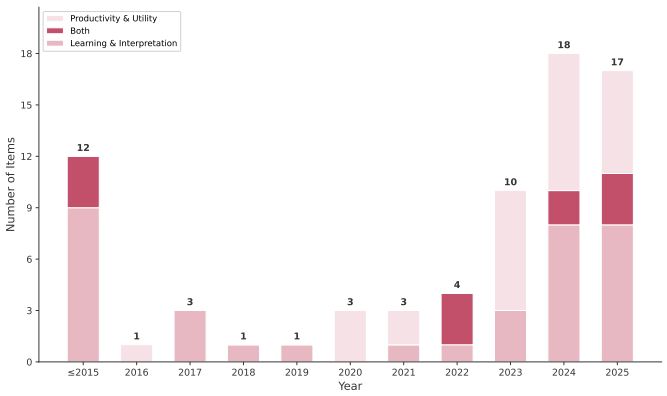


Fig. 3. Temporal distribution of surveyed PTs by Usage & Purpose. The first bin aggregates PTs published before and including 2015.

Snowballing: After the venue and agentic search, we expanded the database by manually reviewing related-work sections, references, and tool documentation. We repeated this process until no additional relevant PTs were found.

Overview Checks and Criteria: The methods above identified candidate PTs, while overview checks filtered candidates before they entered the final database (Fig. B.5). For publications, we used an LLM agent based on Gemini 2.5 Flash [31] to extract titles, abstracts, authors, and summaries into overview documents for manual filtering. For tools, we summarized websites and documentation into overview documents focused on visualization-related features. The prompts are provided in Listings B.1 and B.2. Based on these overview documents, we manually filtered candidates using three criteria: (1) The candidate had to present or provide visualization as a substantive part of its approach or tool functionality, including graphical interfaces, visual interactions, or plotting. (2) It had to target QC, rather than quantum physics. (3) It had to support a QC use case, such as helping users analyze, debug, or develop quantum states, circuits, hardware, or algorithms. This excludes work in the opposite direction, where QC is used as a computational method for visual-data tasks. Filtering details are provided in Appendix B.

Additionally, to avoid hallucinations in generated overview documents, we skimmed the filtered-out publications from these three perspectives before excluding them. We found no publications misclassified as irrelevant and no hallucinations in overview checks. Together, the three collection methods and overview checks provide comprehensive coverage of relevant VIS4QC research, yielding a final collection of 69 PTs.

B. Coding of PTs

Our analysis has two stages. In the first stage, we read all overview documents of the core PTs, tagging them with applications, purposes, quantum concepts, and domains. At this stage, we did not code visualization types or patterns. Because existing VIS4QC studies are sparse and visualization can be characterized along many dimensions, imposing visualization categories too early would make the coding unstable. We therefore first coded user goals and QC subjects, and later analyzed visualization patterns as the third taxonomy dimension described below. Our goal was to answer two key questions first: *"What user goals do the visualizations serve?"* and *"What QC subjects do they address?"*

Two co-authors independently coded all PTs with their own codes. To ensure consistency in quantum terminologies, they referred to the glossary in Nielsen and Chuang [32]. They then discussed their codes to reach an initial consensus. Subsequently, they discussed the codes and categories with QC expert co-authors during weekly meetings, iterating over multiple rounds until the author team reached final consensus.

In the second stage, the two authors independently coded the core PTs again to validate the final taxonomy. They compared their code assignments. When assignments differed, they discussed the discrepancies, refined the code book, and recoded the PTs until they reached consensus. Appendix B-1 presents representative borderline cases and explains how the refinements to the code book resolved the discrepancies. The coding categorizes PTs along the first two taxonomy dimensions: Usage & Purpose of Visualizations (*why*; Section IV) and QC Subjects (*what*; Section V). The third dimension, namely *how* existing work visualizes QC subjects (Section VI), was developed by analytically reviewing the visualization techniques employed across the core PTs.

The codes of all core PTs are summarized in Table 1. Some PTs may have multiple tags, as they provide comprehensive solutions to multi-faceted problems. We further distinguish between PTs that provide only conventional visualizations versus those that contribute novel or advanced visualization designs. In Table 1, colored marks (squares, triangles, circles) indicate category assignments. Triangles indicate that the work provides only conventional visualizations, including the Bloch Spheres [14], bar charts, and standard circuit diagrams. Works with squares contribute more tailored visual designs. As Quantum Error Correction (QEC) requires distinct QC knowledge and visualizations, we mark related PTs with a circle. Industry tools are highlighted in indigo at the bottom. Fig. 2 presents the code assignment distribution of PTs by QC subject and purpose, while Fig. 3 illustrates the temporal distribution.

IV. Usage & Purpose of Visualizations

Here, we discuss the first dimension of our taxonomy: *why* a visualization is created. Visualizations in QC serve distinct purposes, namely *Learning & Interpretation (LI)* and *Productivity & Utility (PU)*. Visualizations for *LI* teach QC concepts or provide novel visual representations that offer

fresh perspectives on established concepts. They generally require little prior QC knowledge and are common in education and research. Visualizations for *PU* help users accomplish tasks (e.g., debugging and machine selection) more efficiently in professional settings and further improve task outcomes, such as lower training loss or higher model accuracy. They assume familiarity with the specific tasks involved. As shown in Fig. 3, *LI* have accumulated a large body of research over decades since the Bloch Sphere [14]. There are limited studies on *PU* before 2023, and most of them were developed thereafter. Table 2 summarizes two main categories along this dimension. Note that a work may serve both purposes simultaneously. We detail the characteristics of each category in the following Section IV-A and IV-B.

TABLE 1
CODINGS OF PTS. ■: CATEGORY ASSIGNMENTS, ▲: ONLY CONVENTIONAL VISUALIZATIONS, ●: QEC-RELATED WORK.

PTs	Venue	Year	QS	GC	MS	AD	LI	PU
Bloch Sphere [14]	Phys. Rev.	1946	■				■	
Walck et al. [25]	Eur. J. Phys.	2001	■				■	
Decision Diagrams for QC [26]	ASP-DAC	2003	■	■		■	■	
Karafyllidis et al. [33]	QIP	2003					■	
ZX Calculus [27]	ICALP	2008	■	■		■	■	■
Altepeter et al. [34]	CLEO/IQEC	2009	■				■	
Mäkelä et al. [28]	Phys. Scr.	2010	■				■	
Fractal Representation [29], [30]	ICQNM & SysMea	2011 & 2012	■				■	
Tahan et al. [35]	APS March Meeting	2015	■	▲			■	
Chernega et al. [36]	J. Russ. Laser Res.	2017	■				■	
QuaSim [37]	IEEE EIT	2017				■	■	
ShorVis [38]	ICVRV	2017	▲	▲		■	■	
QuFlow [39]	SciVis	2018		■			■	
Rainbow Boxes [40]	GRAPP	2019	■				■	
GraphStateVis [41]	IEEE QCE	2021	■				■	
Arawjo et al. [42]	ACM UIST	2022		■	■		■	■
Jordon et al. [43]	IEEE QCE	2022				■	■	
Suau et al. [44]	IEEE QCE	2022	■				■	■
qprof [45]	ACM TQC	2023		■				■
QWalkVis [46]	IEEE QCE	2023				■	■	
Quantivine [47]	TVCG	2023		■				■
Senapati et al. [48]	QCCC	2023	■				■	
VACSEN [49]	TVCG	2023	■	■	■			■
VENUS [50]	CGF	2023	■				■	
Bley et al. [51]	Phys. Rev. Res.	2024	■				■	
Chou et al. [52]	IEEE QCE	2024	■	▲		■	■	
CircInspect [53]	IEEE QCE	2024		■				■
HammingVis [54]	Graph. Models	2024	■	■			■	■
Kim et al. [55]	IEEE QCE	2024	▲	■	■			■
QGrover [56]	IEEE QCE	2024	▲	▲		■	■	
QML Playground [57]	IEEE CG&A	2024	■			■	■	
QNotation [58]	IEEE QCE	2024	▲	▲			■	
QuantumEyes [59]	TVCG	2024	■	■			■	
QubiCSV [60]	Sci. Rep.	2024			■			■
QVis [61], [62]	IEEE QCE & ISVLSI	2023 & 2024	■	▲	■			■
Seidel et al. [63]	arXiv	2024	■			■	■	
VIOLET [64]	TVCG	2024	■	▲		■	■	
BEADS [65]	New J. Phys.	2025	■	■			■	
Intuit [66]	ACM CHI	2025	■	■			■	
Kim et al. [17]	ACM CHI	2025	■	■	■			■
MuqcsCraft [67]	ACM TQC	2025[b]	■	■			■	
QCanvas [68]	IEEE QSW	2025	▲	▲			■	■
Patel et al. [69]	IJAMRS	2025	■			■	■	
VisQI [70]	CGF	2025[b]		▲		■		■
QuRAFT [71]	TVCG	2025	▲	▲		■	■	
state-o-gram [72]	arXiv	2025	■	▲			■	
Sanderson [73]		2025	■			■	■	
Scruby [74]	arXiv	2025		●			■	
Senapati et al. [75]	IEEE QSW	2025	■					■
XQAI-Eyes [76]	TVCG	2025[b]	■	■			■	■
QuTiP [77]–[79]	CPC & Phys. Rep.	2012[b]	■				■	■
pyQuil [80]	arXiv	2016	▲	▲				■
Cirq [81]	Zenodo	2018[b]	▲	▲	■			■
Braket [12]		2020[b]	▲	▲				■
qecsim [82]	PhD Thesis	2020		●				■
TKet [83]	QST	2020		■		■		■
Qibo [84]	QST	2021	▲	▲				■
Stim [85]	Quantum	2021		●				■
PennyLane [11]	arXiv	2022		■			■	■
qBraid [86]	Zenodo	2022[b]	▲	▲				■
Cuda Q [13]	DAC	2023	▲	▲				■
Q# [87]	EPTCS	2023	▲	▲	■			■
Quantum Code Visualizer & PanQEC [88]	PRX Quantum	2023		●				■
Bloqade [89]	Zenodo	2024	▲	▲				■
Qadence [90]	IEEE Software	2024[b]		▲				■
Qiskit [10]	arXiv	2024	■	■	■		■	■
Qrisp [91]	arXiv	2024	■	▲		■		■
Qualtran [92]	arXiv	2024		■				■
Triple Alpha [93][a]		2025			■			■

[a] Due to limited access to it, we only checked the website of Triple Alpha
[b] Year of first public appearance, announcement, or release.

TABLE 2
Usage & Purpose CATEGORIES IN VIS4QC: THEIR GOALS, APPLICATIONS, AND PREREQUISITE KNOWLEDGE.

Aspect	*Learning & Interpretation*	*Productivity & Utility*
Goals	• Support learning through explanatory examples, tutorials, or pedagogical interfaces • Provide new visual representations for interpreting QC subjects	• Improve user productivity by presenting useful data visually, which can in turn lead to better task outcomes and performance
Application Domains	• Education • Physics and QC Research	• Software Engineering • QC Research • System Management
Prerequisite Knowledge	• None or basics	• Complete understanding of specific tasks

A. *Learning & Interpretation of QC Knowledge*

Visualizations in this category serve two related goals, which we separate by their primary role and audience. Teaching-oriented work uses visualization as pedagogical scaffolding for learners. Interpretation-oriented work proposes a visual representation as a lens for researchers to reason about a QC subject. The boundary is not absolute, but we classify works based on their main contribution and target audience.

Teaching-Oriented Visualizations: In quantum education, teachers use visualizations to make quantum concepts and algorithms more accessible. For example, *Intuit* [66] (Fig. C.11) uses visual analogies to help students understand basic quantum concepts (e.g., superposition and measurement). *QNotation* familiarizes learners with different notations used in QC. Arawjo et al. [42] develop a drawing-based interface to teach quantum programming. Various visualizations [33], [38], [43], [52], [56], [69], [71], [73] are also developed to help learners understand canonical quantum algorithms in QC textbooks. Educational quantum games [22], [23], [35], [37], [94]–[97] serve the same purpose but emphasize engagement and motivation, embedding visualizations within game mechanics to sustain learner interest. Because the goal is to build understanding, these visualizations typically focus on small-scale quantum systems where the underlying phenomena remain tractable enough to explain visually.

Interpretive Representations: Beyond teaching, some visualizations offer researchers graphical representations that reveal structures not apparent from mathematical formalisms alone. These studies often stem from physics and QC research. One canonical example is the Bloch Sphere [14], which gives a single-qubit state an intuitive geometric identity as a point on a unit sphere, making abstract operations such as rotations and phase shifts directly visible (Appendix A-1.1). Since then,

numerous physicists and mathematicians have developed representations that extend this geometric interpretation to more quantum systems. For example, the fractal representations proposed by Galambos et al. [29], [30] reveal the internal structure of multi-qubit systems by exploiting self-similarity: each qubit appears as a building block of the fractal (Fig. C.14), making it possible to simultaneously see individual qubit properties and their entanglement. Chernega et al. [36] offer a complementary perspective on single-qubit states by mapping them onto the vertices of a triangle, establishing a one-to-one correspondence with points inside the Bloch Sphere. This reframing makes uncertainty relations for measurement probabilities geometrically visible as constraints on the triangle's area. Mäkelä et al. [28] apply the Majorana construction to pure N-qubit states, yielding $2^N - 1$ points on the Bloch Sphere. Despite their potential applications in education, these visualizations are mainly used by physics and QC researchers to interpret properties of QC subjects. Compared to purely educational visualizations, they require a basic understanding of the target QC subject.

B. Productivity & Utility

Visualizations in this category target expert users who need to work more efficiently or achieve better task outcomes. They need to support professional workflows in realistic settings, rather than conceptual demonstrations. Most existing PTs focus on *productivity*: helping users accomplish a task faster or with less effort. However, some also yield *utility* gains by improving the quality of the outcome.

Productivity: A number of SDKs (e.g., Qiskit [10] and PennyLane [11]) already help users program, debug, and execute quantum circuits on simulators or hardware, and produce basic visualizations, reducing the manual effort required at each stage of the development workflow. The PTs surveyed here address productivity bottlenecks. For example, selecting a quantum computer with a favorable noise profile is critical for computational fidelity. Yet, the noise landscape is complex and changes over time, forcing users into a time-consuming trial-and-error process. *VACSEN* [49] (Fig. C.18) replaces this manual search with an interactive visual comparison of noise across machines and compiled circuits. *qprof* [45] (Fig. C.23) helps users identify performance bottlenecks and optimize their implementations through visual profiling reports. *QubiCSV* [60] helps users rapidly analyze the vast amounts of data generated by qubit control systems. Beyond these specific bottlenecks, several recurring productivity needs span the QC development workflow. For example, verifying and debugging state transitions remain largely manual, as developers must trace how intermediate states change at each step. More broadly, Kim et al. [17] survey ten open-source QC tools, identify recurring usability barriers that slow developers down, and propose interaction techniques that streamline tasks from circuit authoring to result analysis.

Utility: Some PTs also improve *utility*; that is, they enhance the quality of task outcomes rather than merely reducing time or effort. In current work, however, utility gains usually arise alongside productivity improvements rather than as independent design objectives. For example, *VACSEN* [49] not only facilitates machine selection but also helps users identify less noisy configurations, thereby improving computation fidelity. Similarly, *XQAI-Eyes* [76] helps users efficiently explore quantum encoder choices for Quantum Machine Learning (QML) tasks and identify encoders with lower training loss. Visualizations designed to optimize task outcomes directly remain an open question (Section VIII-D).

V. QC Subjects

This section addresses the second dimension of our taxonomy: *what* QC subjects are visualized. We organize these subjects by *abstraction level* because subjects at similar levels share data forms, tasks, and visualization challenges. We first present the rationale for this organization and then survey related studies in the following subsections.

Because QC spans hardware platforms, software stacks, algorithms, and applications, there is no single field-wide taxonomy that fits the needs of VIS4QC. Broad references instead discuss the field through coarse divisions such as hardware, algorithms, and application domains [3], [4], [32], [98]. These divisions are useful for introducing QC, but too coarse for VIS4QC because they do not distinguish the visualized subjects that shape visualization design, such as states, circuits, machine properties, and algorithms. To comprehensively map out the relevant QC landscape and inform future exploration in VIS4QC, we classify VIS4QC research by QC subjects organized by abstraction level. We first separate the surveyed work into two broad parts: *Quantum Computer Implementation (QCI)* focuses on understanding, building, and optimizing quantum computers, while *Algorithm Development (AD)* focuses on using quantum computation to solve problems in various use cases and applications. Fig. 4 summarizes these QC subject categories. Together, these two parts cover the surveyed VIS4QC work because each visualization either targets the construction, operation, or analysis of quantum computers or the use of quantum computation to solve application problems.

Within *QCI*, we organize subjects into three categories: *Qubits & States* (Section V-A1), *Gates & Circuits* (Section V-A2), and *Machines & Systems* (Section V-A3). These follow commonly used building blocks of quantum computers and align with visualization tasks because they determine whether a design visualizes states, transformations, program structure, hardware properties, or system behavior. We do not subdivide quantum algorithms into finer classes because (1) they are emerging quickly, (2) the taxonomy of quantum algorithms (e.g., [4], [5]) is complex and remains subject to change, and (3) existing VIS4QC studies for quantum algorithms are sparse.

Note that a VIS4QC study can span multiple QC subject categories, as a challenge may involve multiple subjects.

A. Quantum Computer Implementation

A significant body of QC research focuses on implementing quantum computers, and most existing VIS4QC research addresses implementation-focused subjects, as shown in Fig. 2. The details of quantum computer implementations can be

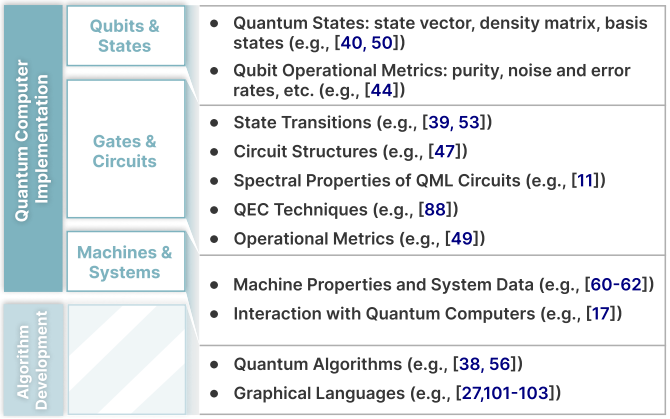


Fig. 4. Table of QC subjects. We separate work on *Quantum Computer Implementation* from *Algorithm Development*. Implementation-focused work is further organized into *Qubits & States*, *Gates & Circuits*, and *Machines & Systems*; each category lists the included subjects.

organized into three categories: *Qubits & States (QS)*, *Gates & Circuits (GC)*, and *Machines & Systems (MS)* (Fig. 4).

1) Qubits & States: Qubits are the fundamental information units of QC (Appendix A-1). The qubit abstraction describes quantum information without requiring every visualization to expose the full physical implementation details of the underlying system. *QS* includes the following key qubit attributes.

Quantum States: Quantum states are the central subject in *QS*. We also include measurement outcomes and their distributions as quantum-state data. For computational-basis measurements, observed bitstrings identify basis states, and their frequencies estimate the corresponding measurement probabilities. Existing quantum state visualization shows state vectors, density matrices, and/or the attributes (e.g., measurement probabilities, phase, and amplitudes) of basis states. For example, the Bloch Sphere [14] visualizes the state vector of a single-qubit state, while the phase disk [99] (Fig. C.6) visualizes the phase, the probability of basis state $|1\rangle$, and the purity of the state. For multi-qubit states, different studies target different aspects of a multi-qubit state due to the scalability challenge (Section VI-B2). For example, *Rainbow Boxes* [40] visualizes measurement probabilities and relative phases of basis states. *VENUS* [50] (Fig. C.7) focuses on measurement probabilities and their normalization condition. *XQAI-Eyes* [76] targets similarities and differences among quantum states encoded from data samples and presents their distributions based on their density matrices.

Qubit Operational Metrics: Beyond quantum states, qubit-level operational metrics, including purity, noise, and error rates, form another data class in this category. For example, Suau et al. [44] focus on the quality of qubit implementations on a quantum computer. *QVis* [61], [62] targets the temporal evolution of qubit noise levels.

2) Gates & Circuits: The *GC* subjects describe how quantum operations and noise transform quantum information during computation. Gates and measurements are atomic operations, while circuits compose them into executable programs. *GC* includes state transitions, circuit structures, spectral properties of QML circuits, QEC-related structures at the circuit level (Appendix A-5), and related operational metrics.

State Transitions: State transitions capture how quantum states change under quantum operations or noise. The relevant data that has been visualized includes the intermediate basis-state parameters (e.g., probabilities and phases) at every step in a circuit's execution. For example, *QuFlow* [39] represents the complete parameter flow of all 2^N possible outputs through a circuit, showing how each gate transforms the probabilities and phases of every basis state. Its 2^N-row layout intrinsically limits visual scalability, as discussed in Section VI-B2. *CircInspect* [53] lets users inspect program outputs, such as measurement probabilities, at user-defined breakpoints while exploring circuit structure at gate and subroutine levels.

Circuit Structures: A circuit model represents a sequence of quantum operations (e.g., gates and measurements) applied to qubits. Because quantum gates are unitary, quantum circuit structures have a distinctive property: qubit lines run strictly in parallel without fan-in or fan-out, unlike classical circuits, where wires can merge and branch. Circuit diagrams are the standard visual notation for these circuit models. Beyond circuit diagrams, researchers have developed more compact circuit representations. *Quantivine* [47] (Fig. C.8) visualizes large circuits through hierarchical sub-circuit grouping and repeated gate-pattern summaries. It also adds context such as qubit provenance, gate placement, parallelism, connectivity, and entanglement. Additionally, Qiskit [10] and Qualtran [92] represent circuits as directed acyclic graphs (DAGs), exposing computational dependencies between operations that are implicit in standard circuit diagrams.

Spectral Properties of QML Circuits: The spectral structure of QML circuits, i.e., the Fourier frequencies and coefficients that characterize a circuit's representational capacity, forms another *GC* subject. PennyLane [11] implements the Fourier analysis proposed by Schuld et al. [100] and visualizes these spectral components.[1]

QEC Techniques: QEC techniques, such as stabilizer codes (Appendix A-5), operate at the circuit level and introduce their own data structures (e.g., code lattices, stabilizer groups, and syndrome patterns). Dedicated visualization approaches remain sparse. Existing PTs include Stim [85] and its interactive editor Crumble for stabilizer circuits, PanQEC [88] for topological codes, qecsim [82] for Pauli operators, and Scruby [74] for quantum product codes.

Operational Metrics of Gates & Circuits: Operational metrics of gates and circuits, such as gate fidelities, circuit depths, and gate counts, form another data class. For example, *VACSEN* [49] (Fig. C.18) shows circuit-level noise profiles across different quantum computers and compares compiled physical circuits for the same algorithm.

3) Machines & Systems: *MS* covers the highest layer of *QCI*, including the physical machine, its supporting infrastructure, and the human interface to the system. We also include QHCI studies in this category, as they concern the interaction between humans and quantum systems as a whole.

[1] PennyLane documentation on Fourier Analysis

Machine Properties and System Data: These data describe the device and its supporting infrastructure rather than quantum states or logical circuits. We classify qubit- and gate-related measurements when they are used for device characterization, calibration, control, or selection. Key data include hardware connectivity, calibration and control parameters, characterization metrics, and temporal system behavior. For example, *QVis* [61], [62] covers the hardware connectivity topology of a quantum computer alongside characterization metrics (e.g., relaxation time, decoherence time, readout error rate), their temporal trends, and inter-qubit similarity in terms of these metrics. *VACSEN* [49] (Fig. C.16) covers noise attributes across multiple quantum computers, together with gate-level error rates in circuits. For physical machine data, *QubiCSV* [60] visualizes calibration and control system data, including qubit calibration parameters (e.g., drive frequency, readout frequency), gate-control configurations (e.g., phase, amplitude, envelope), and characterization metrics (e.g., readout fidelity and coherence times).

Interaction with Quantum Computers: QHCI studies address the interaction design between users and quantum computers. Arawjo et al. [42] address quantum circuit structure authoring, including gates, sub-circuit definitions, and qubit bundles. This is done through a pen-based notation that supports abstraction features such as custom gate definitions and recursive circuit composition within Jupyter notebooks. Kim et al. [17] survey existing QHCI studies and propose interface techniques that span the full QC development workflow. These techniques cover circuit composition (linking high-level problem concepts to gate-level circuits), machine selection (gate-qubit error rates and time-serial machine data), circuit optimization (logical versus physical circuits, fidelity information), and result analysis (measurement outcomes, problem-specific outputs, and uncertainty).

B. Algorithm Development

AD emphasizes the conceptual logic and mathematical structure of quantum algorithms. This includes visualizations designed to explain or analyze particular algorithms. Using an algorithm solely to demonstrate a general-purpose state or circuit representation does not by itself warrant *AD*. Rather than subdividing quantum algorithms into finer classes, we discuss two recurring subject types in the surveyed studies: (1) Quantum Algorithms covers visualizations for named algorithms and the intermediate values produced during their execution. (2) Graphical Languages covers diagrammatic notation systems for reasoning about composition, equivalence, and transformation across quantum computations.

Quantum Algorithms: Most existing *AD* visualizations address canonical quantum algorithms commonly used in QC textbooks and courses, such as Grover's Algorithm [2], Shor's Algorithm [1], and Quantum Fourier Transform. The primary data are intermediate values produced during algorithm execution, including probabilities, state vectors, and amplitudes, which are typically presented in the sequential order of algorithmic steps. A notable example is the video by Sanderson [73] on Grover's Algorithm: it explains the algorithm through state vectors and their geometric relationships rather than through gates or circuits. Interactive systems also allow users to inspect how these quantities change across algorithmic steps (Table 3). The amplitude visualization tool presented by Chou et al. [52] shows changes in state amplitudes as users step through Grover's amplitude amplification, while *ShorVis* [38] supports forward and backward navigation through Shor's algorithm to inspect intermediate register states. Interactive systems also address QML [57], [64]. Beyond these examples, *Qrisp* provides tree diagrams for inspecting quantum backtracking at successive algorithmic steps [63], [91] (Fig. C.22).

TABLE 3
SEQUENTIAL VISUALIZATIONS FOR CANONICAL QUANTUM ALGORITHMS AND INTERMEDIATE QUANTITIES VISUALIZED DURING EXECUTION. PROBABILITIES (P), STATE VECTORS (SV), AND AMPLITUDES (A).

PTs	Algorithms	P	SV	A
Chou et al. [52]	Grover's Algorithm [2]	■	■	■
Patel et al. [69]	Grover's Algorithm [2]	■		
QGrover [56]	Grover's Algorithm [2]			■
Sanderson [73]	Grover's Algorithm [2]		■	
QuRAFT [71]	Few-qubit Quantum Algorithms, including Grover's Algorithm [2]	■	■	
Karafyllidis et al. [33]	Quantum Fourier Transform	■		
QWalkVis [43], [46]	Quantum Walk	■		
ShorVis [38]	Shor's Algorithm [1]		■	

Graphical Languages: Rather than targeting a specific algorithm, graphical languages provide diagrammatic rules for composing, transforming, and proving equivalence between quantum computations. These rules support algorithm analysis by helping users reason about how states, gates, and circuits compose and transform. These visualizations present graphical symbols and their composition according to a notation system. The resulting expressions can represent states, gates, or entire computations. These representations can expose algebraic and compositional structures that are difficult to see in standard circuit diagrams. The ZX-calculus [27], [101]–[103] is a rigorous graphical language that represents quantum computations as tensor networks, generalizing Z and X operations to reveal such structures. Decision diagrams [26], [104], [105] (Fig. C.20) offer compact graph-based representations of states and gates that capture their algebraic structure.

VI. VISUALIZATIONS FOR QC

The third dimension of our taxonomy focuses on *how* QC subjects are visualized. This section focuses on the visual forms and data-specific strategies used in existing VIS4QC work. Section VI-A first frames visual representations along three axes: *dimensionality*, *customization*, and *visual representation type*. For readability, we present the literature by dimensionality first, then distinguish standard charts from customized forms such as spheres, node-link diagrams (NLDs), treemaps, and 2D glyphs. Section VI-B1 then classifies the target data into computational and operational types. Building on these two views, Section VI-B2 and VI-B3 analyze how visualization choices respond to data-specific challenges.

A. Visual Representations

Existing VIS4QC work uses a diverse set of visual forms, ranging from familiar charts to customized representations designed around quantum states, circuits, and hardware. We classify visual representations in VIS4QC along three axes. The first axis is *dimensionality*: whether a design uses 3D or 2D visual forms. Here, dimensionality refers to the visual representation space, rather than the underlying data domain. We distinguish planar representations (2D) from those using a third spatial dimension to position, extend, or arrange visual elements (3D). In this paper, height-based and layered forms, often termed 2.5D [106], [107], are also included in our 3D category. The second axis is *customization*: whether a design directly uses standard charts or introduces customized visualizations for QC data. Standard charts refer to familiar chart types, such as bar charts, line charts, scatterplots, and heatmaps, that keep their conventional layouts and marks when applied to QC data. By contrast, customized visualizations introduce quantum-specific spatial layouts, metaphors, or glyphs. For customized visualizations, the third axis is *visual representation type*, such as spheres, 3D metaphors, NLDs, treemaps, or 2D glyphs. For presentation clarity, we organize the following discussion by dimensionality first, then distinguish standard charts from customized representation types within each dimensional setting. Fig. 1 provides a summary of visual representation types.

1) 3D Representations: 3D designs in existing work include both standard charts and customized visualizations. Standard 3D charts directly extend familiar chart forms to selected quantum values. Customized 3D forms include spheres that encode quantum-state geometry and metaphors that explain quantum concepts. While 3D designs often align with geometric interpretations of quantum states, they introduce common 3D visualization limitations: occlusion, perspective distortion, and difficulty reading precise values [108], [109].

3D Bar Charts: Some 3D designs use standard charts for selected quantum values. For example, Qiskit's cityscape plot represents density-matrix elements as 3D bars on a grid [10].

Spheres: Sphere-based designs primarily represent quantum states, originating from the canonical Bloch Sphere [14]. The Bloch Sphere maps a single-qubit pure state to a point on a unit sphere using two angular parameters (Appendix A-1.1). Mixed states appear as points inside the sphere. The Bloch Sphere can also show the effects of quantum errors such as bit flips, phase flips, and decoherence [110]. *BEADS* [65] uses colored spheres to represent individual qubit states and correlations among qubits, helping readers understand measurement probabilities and entanglement. Several systems extend this spherical mapping to multi-qubit states by reparameterizing the state space. *Q-sphere* [10] arranges basis states on a sphere by Hamming weight, encoding probability as point size and phase as color. Mäkelä et al. [28] similarly use the Majorana construction to map pure N-qubit states to $2^N - 1$ points on the Bloch Sphere. This gives a geometric view of pure multi-qubit states as point configurations on a familiar spherical surface.

3D Metaphors: Some customized 3D designs use familiar objects as metaphors for explaining quantum concepts. For example, *Intuit* [66] (Fig. C.11) uses rotating coins, cards, and spinners to represent concepts in QC (e.g., superposition and entanglement). The mapping is metaphorical: for example, the number of objects corresponds to the number of qubits and rotation represents state duality.

2) 2D Representations: 2D designs project QC information onto a flat plane without the occlusion and perspective distortion of 3D layouts. We organize these designs by customization: standard charts are often used directly, while customized visualizations adapt relational, hierarchical, or glyph-based forms to quantum states, circuits, or hardware metrics. We discuss standard charts first, followed by customized NLDs, treemaps, and glyphs.

Standard Charts: In 2D representations, standard charts are widely used to display projected quantum information, show distributions, and compare metrics. For instance, *state-o-gram* [72] (Fig. C.17) positions bars horizontally by phase angle and stacks them vertically in computational-basis order, with each bar's height encoding the corresponding measurement probability. For high-dimensional data, *XQAI-Eyes* [76] visualizes projected quantum state distributions for QML applications in a scatter plot after Principal Component Analysis (PCA) on density matrices. Other systems use standard charts to track performance and optimization metrics. *QVis* [61], [62] (Fig. C.25) combines heatmaps, line, and bar charts to show calibration trends, metric distributions, qubit similarity, and circuit optimization results. Kim et al. [17] uses standard charts and tables to visualize layer-wise and cumulative estimated success probabilities for circuit fidelity. *QubiCSV* [60] uses line charts for qubit and gate calibration properties.

Node-link diagram (NLD): NLDs represent items as nodes and relationships as links. In VIS4QC, they are used when the represented subject has an explicit relational structure, including basis-state adjacency, qubit correlations, state transitions, circuit dependencies, and hardware connectivity.

For quantum states, nodes can represent basis states or qubits. Links encode the relation chosen by the visualization, such as one-bit adjacency between basis states, membership in a qubit-correlation subset, or a graph-state entangling relation. For example, in *Dimensional Circle Notation* by Bley et al. [51] (Fig. C.19), the NLD shows the N-dimensional hypercube of the computational basis. Each node represents one basis state $|i_N \dots i_1\rangle$ and carries a circle glyph for its complex amplitude α_i. Each link connects two basis states that differ by exactly one bit, thus representing the change of one qubit value while holding the other qubit values fixed. *BEADS* [65] (Fig. C.10) uses a different NLD encoding. Its nodes are qubits, represented as Q-Beads that encode each qubit's reduced state. Links represent correlation subsets among those qubits. For a pairwise relation $\{k,l\}$, a correlation bead is placed as a node on the line between Q_k and Q_l, indicating that the encoded correlation belongs to that two-qubit subsystem. The bead type of a node distinguishes connected entanglement-based correlations (E-Beads), compound correlations (C-Beads), and total correlations (T-Beads). *GraphStat-eVis* [41] (Fig. C.21) visualizes graph states (a special family of multi-qubit states used in QEC) as undirected qubit graphs.

Each node represents a qubit, and each link represents the controlled-Z relation used to prepare the graph state. Changing the NLD topology changes the represented state. Decision diagrams [104] offer another visual interpretation of quantum computations by representing quantum states and operations as NLDs, with nodes representing recursive decompositions, and links carrying complex-valued weights (color for phase and thickness for magnitude).

For quantum state transitions, NLDs represent how states or outcomes change under quantum operations or noise, revealing deeper transition structures (e.g., interference and noise propagation). *QuFlow* [39] uses an NLD with a grid layout: nodes are arranged in rows (basis states) and columns (gates), and connecting links show how amplitudes flow between states with different line patterns for different gate types. *HammingVis* [54] visualizes erroneous measurement outcomes in Hamming space using a force-directed NLD, where nodes are output bitstrings and directed links reveal likely error propagation among neighboring outcomes.

When NLDs are used to represent circuit structures, their nodes represent qubits or quantum gates while the links connect the qubits that are acted upon by a quantum gate. The quantum circuit diagram is a special type of NLD, forming strict grids, unlike general NLDs (Appendix A-2). A few approaches extend the standard quantum circuit diagram. For example, *Quantivine* [47] (Fig. C.8) groups gates into semantically meaningful modules that are visualized with compact representations. *CircInspect* [53] (Fig. C.24) abstracts subroutines into circuit blocks, supporting drill-down from high-level overviews to specific components. Beyond quantum circuit diagrams, DAGs offer a directed NLD representation that makes computational dependencies explicit, which is convenient for circuit analysis and optimization. They represent operations as nodes and data dependencies as directed links, removing the fixed qubit-line layout of circuit diagrams to expose the partial ordering among operations. Qiskit [10] and Qualtran [92] provide DAG-based NLD visualizations.

Moreover, some system properties of a quantum computer are naturally represented by NLDs. For hardware topology, nodes represent physical qubits and links represent two-qubit gate connections. Some quantum processors have fixed coupling maps that determine which qubits can directly interact, forming a graph structure. Both Qiskit [10] and Cirq [81] provide functions to visualize device topology as NLDs.

Treemaps: Treemaps represent multi-qubit states through area-based hierarchical layouts. They are useful when the visualization treats basis states or separable factors as parts of a larger state. *Rainbow boxes* [40] arrange qubits as columns and represent basis terms of separable factors as stacked boxes. Each box spans the qubits involved in the factor, making the factorization structure visible through horizontal extent and vertical subdivision. *Fractal representations* [29], [30] (Fig. C.14) use a stricter self-similar hierarchy. When the qubits are not entangled, the view starts with one qubit's bar and adds the others by placing scaled copies of their bars beneath the previous level's 0/1 segments. For general states, including entangled ones, each column represents one possible combination of qubit values. The columns are ordered by their 0/1 patterns, with adjacent bars of matching bit values and phases merged to form the hierarchy. This layout follows the binary decomposition of basis states, with segment width encoding probability and bar color encoding bit value.

Glyphs: Various studies incorporate customized glyph designs. For example, the *phase disk* [99] (Fig. C.6) combines multiple encodings: filled area for probability, a radial arm for phase, and ring radius for purity. Similarly, Bley et al. [51] use a circle-notation glyph for each basis state. The filled inner-circle radius of the glyph encodes the amplitude magnitude. Its area encodes measurement probability, and the radial line encodes phase. The vector field visualization by Suau et al. [44] maps the Bloch Sphere onto a 2D plane, overlaying heatmaps (for purity or fidelity) with vector arrows (showing deviations from ideal states) to reveal state-dependent errors. The dandelion chart in *QuantumEyes* [59] (Fig. C.13) uses a scatter plot where position encodes complex amplitude (real vs. imaginary) and circle area encodes probability. *VIOLET* [64] (Fig. C.15) uses a satellite chart that arranges basis-state circles in a circular pattern and links them to the corresponding 0/1 positions on qubit axes, with circle and line color encoding basis-state probabilities. Stacked bars at these positions encode each contributing basis-state probability by segment height and the corresponding single-qubit marginal probability by total height. *VACSEN* [49] (Fig. C.16) uses a circuit-like layout in its Computer Evolution View to show changes in qubit and gate properties over time. For a selected qubit metric, circle radius encodes deviation from a reference value, while blue and red distinguish better and worse performance. Connecting line segments show gate topology, with horizontal position encoding gate error rates.

3) Combined Representations: Some studies combine individual visual representation types in practice. A recurring pattern combines state encodings with relational or circuit structures. For example, *BEADS* [65] (Fig. C.10) combines spherical glyphs (BEADs) with NLDs to show inter-qubit entanglement in a multi-qubit state. It also overlays BEADs on quantum circuit diagrams to show quantum states at different circuit steps. Similarly, IBM Quantum Composer [99] can place phase disks on circuit wires, combining single-qubit state encodings with circuit structures.

B. Visualizations by Data Type

Visual representations describe the form of a design, while data types shape what a visualization must preserve, reduce, or emphasize. We therefore organize the rest of this section by the data being visualized. We first distinguish computational data from operational data, then discuss how existing visualizations address the constraints of each data type.

1) QC Data Types: We categorize QC data into two types: *computational data* that is intrinsic to or directly produced by the quantum computation process, and *operational data* that describes the conditions, resources, execution behavior, and evaluation outcomes around that computation. Computational data includes quantum states, state transitions, and circuit structures. Measurement outcomes and their distributions are

	Computational Data	Operational Data
Qubits & States	Quantum States	Qubit Statistics (e.g., error rates, T1 and T2 times, etc.)
Gates & Circuits	State Transitions Circuit Structures	Statistics of Gates and Circuits (e.g., gate counts, circuit depth, etc.)
Machines & Systems	——	Hardware Properties and Statistics (e.g., connectivity, calibration data, etc.)
Algorithm Development	Sequence of Transformations on Comp. Data	Metrics of Algorithms (e.g., accuracy, fidelities, etc.)

Fig. 5. Mapping between QC subject categories and data types. *Computational data* covers *quantum states*, *state transitions*, and *circuit structures*, while *operational data* include various statistics and metrics.

also classified as computational data within the quantum-state category (Section V-A1). Operational data includes hardware properties, calibration values, topology, runtime behavior, derived circuit metrics, and algorithm-level quality measures.

We cross-reference the subject categories from Section V to subdivide data within each category into the two data types, as summarized in Fig. 5. Within *QS*, *quantum states* are computational data because they carry complex-valued amplitudes that encode superposition and entanglement. Qubit statistics such as error rates and coherence times, by contrast, are operational data because they characterize the physical medium independent of a specific computation. Within *GC*, *circuit structures* define the program logic and how gates compose, and *state transitions* specify how quantum states transform. Derived circuit metrics such as gate counts, circuit depth, fidelities, and execution costs are operational data because they summarize resource use or execution quality rather than directly describing states, transformations, or structure. The *MS* category concerns the physical substrate and environment that host computation, so the visualized data in this category (e.g., qubit connectivity, calibration parameters, and control-system behavior) are primarily operational. Within *AD*, visualizations often reuse computational data from lower subject categories, such as probabilities, amplitudes, state vectors, state transitions, and circuits, to explain how an algorithm works. They may also use operational or evaluation data (e.g., accuracy, fidelity, convergence, success probability, and resource metrics) to assess algorithm outcomes.

The two data types lead to different *visualization constraints and design challenges*. Computational data often contain complex-valued, high-dimensional, and relational structures intrinsic to quantum computation, such as state amplitudes, phase, entanglement, transitions, and circuit dependencies. Its central challenge is scalability: quantum states scale with qubit count, state transitions add gate-step sequences, and circuit structures grow with both qubit lines and gate sequence length. Fig. 6 summarizes this computational data and previews the scaling dimensions analyzed below. Operational data usually consist of real-valued results, categorical metadata, physical links, and temporal records about hardware, software, execution, and evaluation. These data are less constrained by exponential state-space growth, so existing visualizations often reuse established charts, topology views, and compact glyphs. Their main design concern is coordinating metrics, devices, versions, execution conditions, and outcomes for monitoring, comparison, provenance, and assessment.

2) Visualizations for Computational Data: Computational data represents quantum states, their evolution during computation, and circuit structures. The properties of these data shape the visualization challenges and guide the following discussion. For a single qubit, the main design question is how to visually encode state properties such as probability and phase. For multiple qubits, the exponential growth of the state representation requires scalable visualization strategies. Visualizing state transitions adds a gate-step dimension to the state representation. Circuit structures can be represented more directly, but their visualizations must still accommodate growth in both qubit count and gate sequence length.

Quantum States: A single-qubit pure state can be written with two complex amplitudes, but visualizations usually expose a smaller set of interpretable quantities: measurement probabilities, amplitude magnitudes, and relative phases. Mixed-state views may also encode purity, which helps users distinguish pure states from mixtures with identical computational-basis measurement probabilities (Appendix A-1). Because the data size is fixed for one qubit, the main design issue is visual encoding rather than scalability. For N qubits, however, the state vector contains 2^N complex amplitudes. Even a two-qubit state has four complex amplitudes, with associated magnitudes, probabilities, and phases, and the information of interest often lies in joint structures such as entanglement and correlations. Multi-qubit state visualizations, therefore, use different scaling strategies to decide what to preserve, summarize, or omit.

Single-qubit states have a fixed data volume, so visualization design mainly concerns how to encode a compact set of state quantities rather than scaling with qubit count. Existing designs encode probability or amplitude magnitude, relative phase, and, for mixed states, purity. Spherical representations, such as the Bloch Sphere [14], use geometric positions to express the state and align well with common explanations of single-qubit operations, but they inherit common 3D limitations, such as occlusion and perspective distortion. Compact 2D glyphs instead project these quantities onto planar marks. For example, the phase disk [99] encodes probability with filled area, phase with a radial arm, and purity with ring radius.

Multi-Qubit states introduce both a size problem and a structure problem: the dimension of a state vector grows exponentially with qubit count, while the information of interest often lies in joint properties such as entanglement, correlations, and basis-state relationships. We organize existing visualizations into four scaling strategies: *Information Reduction, Per-Qubit Composition, Relational Reformulation, and Hierarchical Organization*. These strategies emphasize different aspects of the state and make different trade-offs.

- *Information Reduction* first selects *partial* information from a multi-qubit state or projects high-dimensional state data into a lower-dimensional space, and then visualizes the processed values with standard charts. It trades completeness for compact summaries, distributions, or projections rather than directly representing the full quantum state. For example, probability bar charts show the measurement distri-

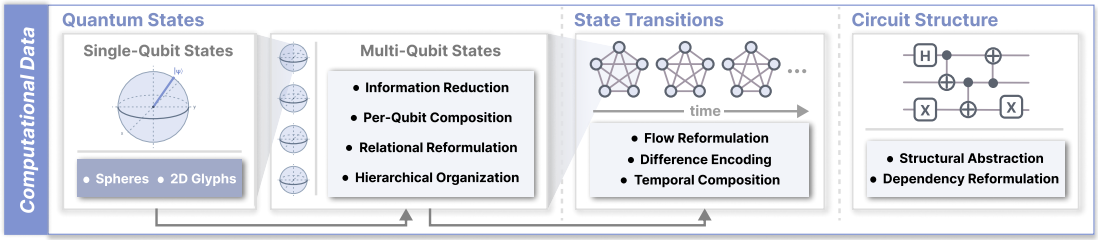


Fig. 6. Visualization techniques for *computational data*, grouped by data type and previewing the scaling dimensions analyzed in the following subsections: qubit count for *quantum states*, gate-step sequences for *state transitions*, and both qubit lines and gate sequence length for *circuit structures*.

bution while omitting phase. Qiskit's cityscape plot [10] can display only the real or imaginary component of a density matrix, providing a partial view of the state. A stronger reduction occurs when the state data are projected before visualization. For example, *XQAI-Eyes* [76] applies PCA to density matrices and visualizes the projected quantum state distributions with a 2D scatter plot. These examples differ in how much state information they retain, from selected values to lower-dimensional distribution patterns.

- *Per-Qubit Composition* composes single-qubit visualizations side by side as small multiples. It scales linearly with N but sacrifices the ability to show inter-qubit relationships, such as entanglement. Phase disks displayed on circuit wires in IBM Quantum Composer [99] exemplify this strategy. Multiple Q-Beads in *BEADS* [65] (Fig. C.10) can be overlaid on a circuit diagram to show the single-qubit states of multiple qubits.
- *Relational Reformulation* formulates multi-qubit states as relations among nodes that represent basis states or qubits, making NLDs a natural visual form. Links then encode a chosen relation between those nodes, such as one-bit adjacency between basis states, membership in a qubit-correlation subset, or a graph-state entangling relation. The emphasis is on the adjacency, correlation, or entangling structure exposed by these links, rather than on individual amplitudes. For example, Dimensional Circle Notation [51] uses a hypercube NLD to show one-bit adjacency among computational basis states. *GraphStateVis* [41] represents graph states as qubit graphs, where links encode the controlled-Z relations that define the state.
- *Hierarchical Organization* views multi-qubit states as hierarchical data, building on the grouping of basis terms within separable factors or the recursive binary decomposition of the computational basis. It maps to treemap-like and fractal layouts, which impose an order on qubits and represent these hierarchies through grouped regions or nested, self-similar structures. *Rainbow boxes* [40] uses a treemap-like layout: qubits are columns, and basis terms of separable factors appear as stacked boxes spanning the qubits involved in each factor. *Fractal representations* [29], [30] recursively subdivide the computational basis by successive qubit values. When the qubits are not entangled, the view starts with one qubit's bar and adds the others by placing scaled copies of their bars beneath the previous level's 0/1 segments (Fig. C.14). For general states, including entangled ones, each column represents one possible combination of qubit values. The columns are ordered by their 0/1 patterns, and adjacent bars with matching bit values and phases are merged to form the hierarchy. The layout follows the binary decomposition of basis states; segment width encodes probability, bar color encodes bit value, and horizontal markers encode phase.

In summary, these strategies trade off completeness, scalability, and interpretability. Information reduction selects, projects, or custom-encodes task-relevant quantities. Per-qubit composition scales linearly but omits joint structure. Relational reformulation emphasizes relationships among basis states or qubits. Hierarchical organization uses grouping and subdivision but depends on qubit ordering and available space. The appropriate strategy, therefore, depends on which aspects of the state must be preserved for the analysis task.

Quantum State Transitions: State-transition data can describe gate-driven evolution or inferred error propagation among measurement outcomes. For gate-driven sequences, it inherits the exponential state-space challenge of multi-qubit states and adds a step dimension: longer circuits create more intermediate states and more changes to compare. Existing visualizations, therefore, differ in how they represent these changes. We organize them into three strategies: *Temporal Composition*, *Flow Reformulation*, and *Difference Encoding*. These strategies respectively compose state snapshots, reformulate transitions as flows or links, or directly encode differences between adjacent states.

- *Temporal Composition* represents a transition process through an ordered sequence of state views. It reuses state encodings and arranges them across gate steps or overlays them on a circuit. *Rainbow boxes* [40] arrange state snapshots sequentially, making it possible to observe how gates transform probability, phase, and entanglement; for instance, seeing a CNOT gate convert single-column separable rectangles into multi-column entangled ones. *BEADS* [65] (Fig. C.10) overlays multiple BEADS representations on circuit diagrams, showing how BEADS evolve over time. This strategy uses small multiples of familiar state encodings, but comparison becomes harder as the number of steps grows.
- *Flow Reformulation* turns state changes into flows or links. It makes movements, splitting, merging, or error propagation visible, but the layout cannot scale with both basis states and gate steps. *QuFlow* [39] uses an NLD with a regular grid layout, where nodes are arranged in rows for basis states and columns for gates; cell color encodes probability, and connecting links show how amplitudes flow between states. Since each basis state occupies a separate row, the layout scales as $T \times 2^N$ for T gate steps and is limited to small

qubit counts. *QuantumEyes* [59] (Fig. C.12) combines an NLD showing how basis-state contributions split and merge across gate operations with a stacked area chart of basis-state probabilities. These coordinated views help users trace how contributions to a selected basis state arise and combine, and relate that process to changes in its probability. For noisy outcomes, *HammingVis* [54] visualizes erroneous measurement outcomes as a force-directed NLD in Hamming space, where nodes are output bitstrings and directed links reveal likely error propagation among neighboring outcomes.

- *Difference Encoding* focuses on what changes between adjacent states rather than showing both states independently. *MuqcsCraft* [67] (Fig. C.9) shows complete state vectors layer by layer and highlights gate-specific amplitude changes with colors, arrows, and symbols. This reduces the comparison burden, but its visual complexity still depends on the number of basis states, which grows exponentially with the qubit count. When difference encoding is paired with mechanism-based abstraction, however, it can express change through a low-dimensional process, such as the rotation view of Grover's Search Algorithm [73].

These strategies trade coverage of state changes for comparison clarity. Temporal composition preserves familiar state views but scales with circuit length. Flow reformulation exposes transition structure but becomes dense with more represented states and longer chains of state changes. Difference encoding focuses attention on changes, but its scalability depends on the abstraction used.

Circuit Structure: Circuit-structure data describes the program skeleton: which gates act on which qubits and in what order. Its scaling is more structurally direct than state or transition visualization. Adding qubits increases the number of wires, while adding gates increases the sequence length. Multi-qubit gates add cross-wire dependencies. The main challenge is therefore not how to encode amplitudes, but how to keep large circuit structures readable while preserving ordering, dependencies, repeated patterns, and qubit provenance.

Existing visualizations use two main strategies to handle this growth. Structural abstraction keeps the quantum-circuit-diagram grammar, while dependency reformulation uses NLDs to draw dependency DAGs.

- *Structural Abstraction* keeps the standard quantum circuit diagram as the baseline representation but compresses large circuits by grouping gates, repeated patterns, or functional modules. *Quantivine* [47] (Fig. C.8) groups gates into functional modules and abstracts repeated vertical, horizontal, or diagonal patterns into compact forms while preserving contextual information such as qubit provenance and gate placement. *CircInspect* [53] abstracts subroutines into circuit blocks, supporting drill-down from high-level overviews to specific components.
- *Dependency Reformulation* recasts circuit structures as dependency graphs or DAGs to expose ordering, parallelism, and critical paths. For example, Qiskit [10] and Qualtran [92] use DAG-based NLDs to help developers analyze circuit dependencies and optimization opportunities beyond the fixed wire-by-time grid.

Together, these strategies address different aspects of circuit growth. Structural abstraction reduces visual size while preserving the familiar wire-by-time layout. Dependency reformulation exposes ordering and optimization structure but moves away from the standard circuit-diagram grammar.

3) Visualizations for Operational Data: Compared with computational data, operational data is usually less quantum-specific in its visual encoding. Most existing systems, therefore, rely on established representations from Section VI-A, especially standard charts and topology-based NLDs, with occasional custom glyphs. The design challenge is not the exponential state-space representation but coordination across metrics, devices, versions, and outcomes.

Specifically, NLDs are used by topology views, where qubits are nodes and hardware connectivity forms links. Qiskit [10] and Cirq [81] provide device-topology diagrams, and Qiskit's `plot_error_map()` overlays gate error rates on the topology. *QVis* [61], [62] links topology views with temporal performance patterns through qubit selection, cross-view filtering, and cluster coloring. Standard charts support monitoring and comparison of various metrics. In QVis [61], [62], these charts complement topology views for analyzing qubit performance and summarize circuit optimization results. *QubiCSV* [60] plots qubit and gate calibration properties across commits. At the algorithm-evaluation level, Kim et al. [17] use charts and tables to summarize derived outcome metrics, including layer-wise and cumulative estimated success probabilities. Some workflows still benefit from domain-specific glyph design. *VACSEN* [49] (Fig. C.16) uses a circuit-like metaphor and custom glyphs to show the temporal evolution of qubit and gate properties, encoding values such as T_1/T_2 and gate error rates. Overall, operational-data visualization can often reuse established approaches. The main challenge is coordinating them for monitoring, comparison, provenance, and outcome assessment.

Taken together, these cases show that data type shapes the visualization problem. Computational data creates scalability pressure through state dimension, transition length, and circuit size, leading visualizations to compose, reformulate, reduce, or abstract data. Operational data scales through devices, metrics, versions, and execution conditions; its challenge is coordinating established views for monitoring, comparison, provenance, and outcome assessment.

VII. Current State of VIS4QC

The current body of VIS4QC work remains limited: our survey identifies 69 core PTs. With a small corpus, aggregate counts alone say little about what kinds of support the field has developed. After Section VI's analysis of how QC data are visually represented, this section turns to where visualization support exists and where gaps remain. We summarize the current state in three complementary views. The first view (Section VII-A) examines where support exists across UP and QCS, the two taxonomy dimensions that define the needs that VIS4QC aims to support. The second view (Section VII-B) examines how these aggregate patterns surface in concrete applications, focusing on quantum education and quantum

software engineering. The third view (Section VII-C) assesses how existing visualization designs support reasoning about quantum concepts and behavior.

A. Coverage Across Usage & Purpose and QC Subjects

Fig. 2 shows support coverage across the UP × QCS grid and separates surveyed PTs by *source* (research publications vs tools). Coverage in each cell is counted by the number of PT assignments. This reveals support-coverage patterns that aggregated counts obscure. We read the grid at three scales: by usage category, by QC subject category, and by individual support cells. Fig. 3 adds temporal context: earlier work mainly supports *LI*, often by explaining quantum states, gates, and canonical quantum algorithms. Since 2023, more *PU* work has appeared, reflecting the maturation of quantum hardware, SDKs, and development workflows. Together, the two figures show a field that is expanding beyond explanation-oriented visualizations but remains uneven across usage purposes and QC subject categories.

Gap Between Academia and Industry: Comparing the two usage categories, *LI* (74 assignments) and *PU* (65 assignments) appear roughly balanced. However, separating research publications from industry tools reveals different emphases. Research publications contribute 69 of the 74 assignments for *LI*, while industry tools contribute only five. For *PU*, research publications contribute 31 assignments and industry tools contribute 34. Academia predominantly focuses on explaining quantum concepts rather than supporting practical tasks. Meanwhile, many industry tools continue to rely on conventional circuit and state views, as indicated by the triangle markers in Table 1. This gap leaves practitioners with limited tooling from academic visualization research, while industry tools incorporate few of the richer visual representations developed in academia.

Thin Coverage in Two Categories: From usage categories to QC subject categories, support coverage is heavily uneven. *MS* (12 total) and *AD* (19 total) are substantially underrepresented compared to *QS* (56) and *GC* (52). The current corpus gives the least visualization support to the categories that are closest to practical applications.

Dominant Support Clusters: Zooming in on individual cells reveals where existing support concentrates. *LI* × *QS* (35), i.e., PTs that visualize qubits and states for learning, and *PU* × *GC* (30) together account for roughly half of all assignments. These two clusters have different origins. Quantum state visualization has roots in physics well before quantum computing became a field (the Bloch Sphere dates to 1946 [14]), giving the *LI* × *QS* cell a long head start. *PU* × *GC*, by contrast, follows the recent growth of modern SDKs, circuit construction, compilation, and debugging workflows, and has grown rapidly since 2023 (Fig. 3). VIS4QC has therefore concentrated where there are familiar objects to visualize: quantum states for teaching and quantum circuits for tool workflows. The missing middle is the bridge between these anchors: fewer systems help learners connect states to circuits and hardware, and fewer tools help practitioners reason about algorithms beyond circuit artifacts.

Algorithms Explained but Not Tooled: The strongest support gap appears within *AD*. *PU* × *AD* contains only 4 PTs, making it one of the least populated cells in the grid. Almost no one is building productivity tools that help experienced users develop, debug, or optimize quantum algorithms. Meanwhile, *LI* × *AD* (15) consists entirely of research publications that explain algorithms visually, with no industry tools represented in this cell.

B. Applications in the Current Landscape

The above patterns become application-specific support needs. We focus on quantum education and quantum software engineering as they are the clearest applications of *LI* and *PU*, respectively.

1) Quantum Education: Quantum education needs visual support that helps learners build connected paths through quantum concepts, not only isolated views of individual artifacts. Education research emphasizes topic ordering, scaffolding, navigation across multiple representations, and connections between theory, simulations, and experiments [21], [111], [112]. This need cuts across QC subject categories, as students move from basic quantum mechanics (QM) concepts to qubits, gates, circuits, algorithms, and hardware. Although several education-oriented systems use visual representations to teach quantum concepts [35], [37], [94]–[97], the surveyed VIS4QC work remains uneven across the topics learners must connect and is rarely packaged for classroom use.

Uneven Learning Pathways: Uneven support creates breaks in the progression across QC subjects. The support grid makes this imbalance visible: *LI* × *QS* is well-populated (35 PTs), whereas *LI* × *MS* contains only 2 PTs. Educators, therefore, have many options for teaching quantum states but little support for connecting those states to hardware behavior. Existing VIS4QC work has developed opportunistically: single-qubit state visualization is well explored, while transitions from noiseless to noisy qubits, simple to advanced algorithms, and conceptual models to real systems remain weakly supported. Visual representations also rarely evolve across these transitions. Different research groups develop incompatible representations for similar concepts, making it difficult to assemble a coherent learning pathway even when individual components exist.

Limited Classroom and Engagement Support: Most education-oriented visualizations remain standalone prototypes developed for specific purposes, with no comprehensive toolkit integrating them. This fragmentation forces teachers to navigate multiple platforms with different interaction and visual styles. As a result, many quantum computing courses rely on mathematical formalism alone, despite evidence that visualization improves conceptual understanding [111]. For example, researchers have proposed rich representations for multi-qubit states (e.g., [28], [30], [50]), yet instructors often fall back to abstract vectors and matrices because these research prototypes are not packaged for classroom use. Quantum games [22], [23], [35], [37], [94]–[97] show promise by offering immediate feedback and gameplay designed to motivate learners. Yet few games use visualization

techniques from the research community, and the scarcity of interdisciplinary talent across game design and visualization persists. Overall, quantum education contains many useful visualizations for individual concepts, but they remain difficult to assemble into a coherent pathway from single-qubit states through circuits, algorithms, and real quantum hardware.

2) Quantum Software Engineering: Quantum software engineering needs visual support that connects conceptual problem formulation, circuit implementation, hardware selection, optimization, execution, and result analysis. Kim et al. identify similar workflow needs and argue that current tools lack integrated interface support, require frequent switching among dashboards, IDEs, documentation, and tutorials, and provide limited support for linking high-level concepts to low-level quantum information [17]. This application assumes experienced users are trying to make quantum programs work on noisy hardware and in hybrid quantum-classical pipelines, which require reasoning across circuits, machines, execution behavior, and outcomes.

Fragmented Workflow Support: VIS4QC tools support individual workflow stages: circuit abstraction [47], [53], profiling [45], noise-aware machine and circuit analysis [49], [61], [62], erroneous-outcome analysis [54], and calibration or control-data inspection [60]. They do not yet provide continuous support across conceptual formulation, circuit design, machine constraints, execution behavior, and task outcomes.

Weak Hardware-Algorithm Reasoning: Developers need to reason about the logic and requirements of quantum algorithms, the capabilities and limitations of quantum hardware, and the strategies for implementing and executing those algorithms on that hardware. These strategies involve decisions such as hardware selection, circuit implementation, error correction, and error mitigation. The algorithm-level visualizations [33], [38], [46], [52] focus on explaining logical computation, while hardware-oriented studies [60]–[62] expose machine properties. *VACSEN* [49] (Fig. C.18) takes a step toward this connection by supporting hardware and compiled-circuit selection for a given algorithm. However, the surveyed work still lacks integrated visual support for reasoning about algorithms, hardware, and the broader implementation strategies connecting them. Developers, therefore, have to piece together information from separate systems to determine how an algorithm should be adapted to the available hardware and which strategies could make its execution more reliable.

Productivity without Outcome Guidance: Current visualizations for quantum software engineering mainly make intermediate artifacts easier to inspect, compare, or debug [45], [47], [49], [53]. Few make task outcomes an explicit design target, such as guiding developers toward higher-fidelity circuits, better backend choices, lower-depth implementations, or more reliable application results. This gap is visible in optimization workflows: Kim et al. note that current tools lack interfaces for comparing optimization outcomes, even though optimization choices can affect circuit fidelity and output quality [17]. Overall, quantum software engineering shows how support gaps become workflow barriers: tools help users inspect artifacts, but few connect them across development stages or guide decisions that improve outcomes.

C. Capabilities and Limitations of Current Designs

The application needs (Section VII-B) require users to interpret quantum behaviors through the conventional and customized representations (Section VI). We examine how current designs support conceptual explanation, expose joint quantum structure, and remain usable as state size grows.

Explaining Quantum Behavior: Existing designs use motion both to enact familiar analogies and to make mathematical transformations visible. *Intuit* [66] (Fig. C.11) uses a rotating virtual coin to illustrate superposition and lets users stop it to illustrate measurement. The coin analogy conveys a transition to a definite measurement outcome, but it does not by itself express the phase relationships that determine interference. Motion can also depict a mathematical transformation: Sanderson's visualization of Grover's Algorithm [73] animates reflections and rotations in a two-dimensional amplitude-space abstraction, showing how the operations change the probability of measuring the target state. With the mathematical relationship made visible, animation can both draw attention to the changing state and explain the operation's effect.

Exposing Joint Quantum Structure: Explaining multi-qubit behavior also requires relationships that cannot be read from local or partial views. For example, the three-qubit Greenberger–Horne–Zeilinger (GHZ) state[2], $(|000\rangle + |111\rangle)/\sqrt{2}$, has genuine tripartite entanglement. When we consider any two qubits and ignore the third, those two qubits are not entangled. A graph whose links indicate entanglement within two-qubit pairs would therefore omit this state's genuine tripartite entanglement. *BEADS* [65] (Fig. C.10) depicts the GHZ state using separate beads for pairwise and three-qubit correlations. Its three-qubit bead exposes a joint property (i.e., the tripartite entanglement) that the pairwise beads alone cannot describe. The design question is whether users can distinguish global entanglement from pairwise correlations. Without that distinction, a visualization may accurately show local states and correlations while leaving the nature of the joint state unclear.

Preserving Interpretability at Larger Scales: Making joint structure explicit introduces further demands on readability as systems grow. The number of possible qubit subsets grows exponentially with qubit count, increasing relationships a design may need to distinguish. Views of quantities associated with individual basis states face a related growth in visual detail, as discussed in Section VI-B2. For example, extending *VENUS* [50] beyond its one- and two-qubit views (Fig. C.7) illustrates this problem. Adding further triangles would accommodate more basis states, each represented by a probability semicircle. Within a fixed display area, representing more equally probable basis states would make these semicircles progressively smaller, making individual values and changes harder to inspect. Scalability therefore depends on whether

[2]Wikipedia article on the Greenberger–Horne–Zeilinger state

users can still compare probabilities, interpret amplitudes, or inspect the relevant relationships at the intended scale.

These examples motivate quantum-specific mappings, composition rules, and interactions as the basis for visual grammars in VIS4QC. Familiar visual forms remain useful when these mappings and rules expose the quantum properties and transformations relevant to the task. A design's adequacy therefore depends on how well it explains the quantum behavior or joint structure relevant to the task, and whether users can still interpret that information as the number of qubits grows.

VIII. Future Directions

Building on the assessment in Section VII, we identify three priorities that we consider promising.

VIS Design for QC: The representation strategies and design challenges discussed in Sections VI and VII motivate visualizations that explain quantum properties and transformations while remaining interpretable as systems grow. Future work could develop visual mappings, abstractions, and interactions that expose relationships and mechanisms relevant to users' tasks, and extend visual support to emerging applications and hardware paradigms. Scalable designs should preserve users' ability to inspect and understand relevant behavior as the amount and complexity of information increase.

AI-Assisted VIS and Visual Analytics: The complexity of QC, the growth of computational and operational data, and the need to reason across workflow stages motivate assistance with creating, choosing, and using visualizations. Generative and multimodal Artificial Intelligence (AI) could help users author or adapt visualizations from task descriptions, circuit code, mathematical expressions, and visual examples. Recommendation could match representations to users' questions and data, for example by suggesting views that expose joint quantum structure when individual-qubit views are insufficient. Summarization could highlight changes relevant to a task across circuit steps or execution runs, with links to the underlying data for further inspection. These activities could build on general AI-assisted visualization methods, guided by the quantum properties and interpretation requirements discussed in Section VII-C.

Bridging Academia and Industry: The assessment in Section VII indicates a gap between visualization research and practice. Future work should investigate how research representations can be incorporated into quantum software tools and connected across development stages, from algorithm formulation to hardware execution and result analysis. Practitioners' tasks and workflow constraints should also inform visualization research priorities. Evaluating these representations within practical workflows could identify which designs improve understanding, support decisions, and contribute to better computational outcomes.

These priorities span QC subject categories, so the following subsections organize more specific research opportunities along this dimension.

A. Qubits & States

For *QS*, future work should focus on state properties that are important for reasoning but difficult to show at scale. Instead of displaying more amplitudes, visualizations should expose phase relationships, interference, and entanglement through task-aware abstractions. This direction follows from the scalability trade-offs summarized in Section VI-B2.

Many current designs make probability visually prominent, while phase is often relegated to color, and entanglement is left implicit or summarized indirectly. This can obscure relationships that are central to quantum computation, especially when interference and correlation explain why a state transformation matters. Better state-property visualizations could help users compare phases, identify entanglement structures, and reason about state changes without enumerating all basis states.

B. Gates & Circuits

For *GC*, future work should move beyond static circuit inspection toward visual support for circuit behavior, transformation, and debugging.

VIS for Dynamic Circuits: Quantum circuits that feature mid-circuit measurements, classical control flow, and conditional operations pose new challenges that existing circuit diagram conventions do not address, as the circuit structure itself becomes dependent on runtime measurement outcomes. As quantum hardware increasingly supports these features, visualization techniques must evolve accordingly.

VIS of State Differences: Gates can be understood through the state transitions they induce, so future circuit-oriented work should represent how operations change quantum states, not only inspect the circuit structure. As discussed in Section VI-B2, difference encoding can reduce the burden of comparing consecutive state snapshots. The remaining opportunity is not only to show numerical differences between states but also to expose the mechanisms behind those differences. For example, applying a Hadamard gate to one qubit of an N-qubit system may change all 2^N amplitudes. Yet these changes arise from a simple operation: applying a 2×2 matrix along the corresponding axis of the quantum-state tensor. Visualizing this compact operation could explain how a local gate changes amplitudes across the state vector, connecting state-transition visualization to mechanism-based algorithm visualization in Section VIII-D.

VIS for Transpilation Understanding: Future work should explain how transpilation (Appendix A-2) rewrites logical circuits for native gate sets, hardware connectivity, and optimization goals. Because multiple physical implementations can differ in depth, fidelity, and resource use, visualizations could help users compare trade-offs, inspect transpilation decisions, and diagnose performance bottlenecks. Such work would address a pain point in quantum software development.

Visual Debugging Tools: Quantum debugging must work under measurement and noise constraints: intermediate states are not directly inspectable on real hardware, measurement collapses states, and phase or correlation errors may not appear in immediate measurement probabilities. For hardware-facing debugging, visualizations could incorporate state estimates reconstructed from measurements across repeated executions. Future visual debugging tools could combine simulator-based

state access, repeated-run statistics, and hardware execution data to help developers identify deviations from expected behavior and trace how errors propagate through circuits. To support fault localization, these tools could link logical circuits, native-gate implementations, and available control-pulse data with simulated or reconstructed state evolution, helping developers narrow down candidate operations and execution intervals, improving fault-localization accuracy and reducing debugging time.

C. Machines & Systems

For *MS*, future work should make hardware behavior easier to interpret and connect to computation.

VIS for Quantum Hardware Statistics: Quantum hardware generates rich operational data, including calibration parameters, noise profiles, cross-talk patterns, and temporal drift. These data are often inspected through calibration experiment plots and dashboards. Interpreting them requires expert inspection across many devices, qubits, and calibration runs, making manual analysis difficult to scale. Visualization can help experts compare hardware behavior, detect abnormal patterns, and reason about how noise affects computation. Recent work by NVIDIA [113] shows that AI models can be trained to interpret such calibration plots. This suggests a concrete future direction: intelligent visual analytics systems that combine visualization with AI agents to help experts inspect calibration data and decide what to tune next.

VIS for Different Hardware Paradigms: New QC hardware paradigms may change not only operational data but also the computational data that visualizations need to represent. Many existing VIS4QC examples assume familiar abstractions such as indexed qubits, circuit diagrams, coupling maps, calibration tables, and gate and readout error metrics. These abstractions transfer across some platforms but not always in the same form. Emerging paradigms such as photonic, neutral-atom, and topological quantum computing may introduce different spatial layouts, native operations, interaction patterns, control constraints, measurement processes, and error models. Some established approaches from Section VI-B3, such as topology views and standard charts, may still transfer, but others will need to be adapted or redesigned around paradigm-specific data. For example, superconducting devices often emphasize fixed qubit connectivity and gate errors, while neutral-atom systems can involve reconfigurable spatial arrangements and interaction patterns. These differences suggest opportunities for visualization techniques that connect hardware-specific constraints with the computational structures they shape.

D. Algorithm Development

For *AD*, the central opportunity is to support complete algorithm development rather than isolated primitives or textbook examples. Future work should connect algorithm mechanisms, hybrid workflows, and outcome-oriented optimization for both quantum education and quantum software engineering.

Mechanism-Based VIS: Current approaches predominantly visualize quantum *states*, i.e., the computational data at different steps of algorithm execution. A complementary direction is to visualize quantum *mechanisms*: the mathematical structures that explain *why* algorithms work, rather than only *what* states they produce. The geometric explanation of Grover's Algorithm discussed in Section VII-C illustrates this approach. Similar ideas could visualize the Quantum Fourier Transform through periodic structures or phase estimation as frequency decomposition. Animation is a natural fit because motion can convey the transformations underlying these mechanisms. However, mechanism-based algorithm visualization remains largely unexplored. Such views are especially useful in quantum education, where learners need intuition about why algorithms work rather than every intermediate state.

VIS for Hybrid Computing: Hybrid algorithms combine quantum subroutines with classical ones. Existing VIS4QC work mainly shows quantum circuits in isolation, leaving the surrounding classical workflow underrepresented. Future visualizations should show how quantum and classical components interact, for example in Shor's classical retry logic.

Utility-First VIS: Future work should help developers improve outcomes from QC, not only inspect intermediate artifacts. They should connect analysis to decisions (e.g., parameter and backend choices, error mitigation strategies, and trade-offs). Utility improvement should demonstrate better results under the same computational budget, or lower total cost for a required result quality while reporting relevant execution and simulation conditions (e.g., shot counts, numerical precision, approximation settings, and failure records).

IX. Conclusion

This survey presents a comprehensive overview of VIS4QC, an interdisciplinary field at the intersection of visualization and quantum computing research. We systematically collected 69 core PTs and proposed a three-dimensional taxonomy that defines the research space of VIS4QC. Our support-coverage analysis shows a clear disconnect between current visualization support and emerging VIS4QC needs. Academia mostly focuses on learning about quantum states, whereas industrial tools mainly support productivity around circuit diagrams. The sparsest areas are *AD* and *MS*, especially productivity support for *AD*, even though algorithm development is central to realizing practical QC applications. These support patterns, along with the distinct visualization challenges across QC subject categories, point to concrete future directions. Examples include scalable visualization of quantum states, visual support for dynamic and transpiled circuits, visual analytics for quantum hardware statistics, and mechanism-based visualization of quantum algorithms. These gaps also surface differently across applications. Quantum education needs connected learning support, classroom integration, and engagement support across QC subject categories. Quantum software engineering needs workflow-level support that links conceptual formulation, circuits, hardware constraints, execution behavior, and task outcomes.

As QC is rapidly advancing, effective visualizations will be necessary to support researchers, developers, and educators. We hope this survey provides visualization researchers with a structured entry point into this emerging field and inspires

future contributions that bridge the gap between visualization innovation and practical quantum computing needs.

ACKNOWLEDGMENTS

Besides the LLM usage in Section III, we also used LLMs to assist us with proofreading and LaTeX programming. However, we wrote and edited the text content manually. This project is supported by the Ministry of Education, Singapore, under its Academic Research Fund Tier 1 (NTU Tier 1, RG104/25) and the Start Up Grant awarded to Yong Wang. Any opinions, findings and conclusions, or recommendations expressed in this material are those of the author(s) and do not reflect the views of the Ministry of Education, Singapore.

REFERENCES

[1] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1484–1509, 1997.

[2] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, ser. STOC '96. Association for Computing Machinery, 1996, pp. 212–219.

[3] A. Montanaro, "Quantum algorithms: an overview," *npj Quantum Information*, vol. 2, no. 1, p. 15023, Jan. 2016.

[4] A. M. Dalzell *et al.*, *Quantum Algorithms: A Survey of Applications and End-to-end Complexities*. Cambridge University Press, 2025.

[5] P. Arnault *et al.*, "A typology of quantum algorithms," 2024. [Online]. Available: https://arxiv.org/abs/2407.05178

[6] A. Ciceri *et al.*, "Enhanced fill probability estimates in institutional algorithmic bond trading using statistical learning algorithms with quantum computers," 2025. [Online]. Available: https://arxiv.org/abs/2509.17715

[7] R. Santagati *et al.*, "Drug design on quantum computers," *Nature Physics*, vol. 20, no. 4, pp. 549–557, Apr. 2024.

[8] Y. Kim *et al.*, "Evidence for the utility of quantum computing before fault tolerance," *Nature*, vol. 618, no. 7965, pp. 500–505, Jun. 2023.

[9] A. D. King *et al.*, "Beyond-classical computation in quantum simulation," *Science*, vol. 388, no. 6743, pp. 199–204, 2025.

[10] A. Javadi-Abhari *et al.*, "Quantum computing with qiskit," 2024. [Online]. Available: https://arxiv.org/abs/2405.08810

[11] V. Bergholm *et al.*, "Pennylane: Automatic differentiation of hybrid quantum-classical computations," 2022. [Online]. Available: https://arxiv.org/abs/1811.04968

[12] Amazon Web Services, "Amazon braket," https://aws.amazon.com/braket/, 2020, accessed: 2026-01-22.

[13] J.-S. Kim *et al.*, "Cuda quantum: The platform for integrated quantum-classical computing," in *Proceedings of the 2023 60th ACM/IEEE Design Automation Conference (DAC)*, 2023, pp. 1–4.

[14] F. Bloch, "Nuclear Induction," *Physical Review*, vol. 70, no. 7-8, pp. 460–474, Oct. 1946.

[15] R. P. Feynman, "Simulating physics with computers," *International Journal of Theoretical Physics*, vol. 21, no. 6, pp. 467–488, Jun. 1982.

[16] Z. Ashktorab, J. D. Weisz, and M. Ashoori, "Thinking Too Classically: Research Topics in Human-Quantum Computer Interaction," in *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. ACM, May 2019, pp. 1–12.

[17] H. Kim, M. J. Jeng, and K. N. Smith, "Toward Human-Quantum Computer Interaction: Interface Techniques for Usable Quantum Computing," in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, Apr. 2025, pp. 1–18.

[18] E. W. Bethel *et al.*, "Quantum Computing and Visualization: A Disruptive Technological Change Ahead," *IEEE Computer Graphics and Applications*, vol. 43, no. 6, pp. 101–111, Nov. 2023.

[19] E. W. Bethel, R. Van Beeumen, and T. Perciano, "Quantum computing and visualization research challenges and opportunities," *IEEE Computer Graphics and Applications*, vol. 46, no. 2, pp. 120–130, 2026.

[20] L. Doyle, F. Seifollahi, and C. Singh, "Building bridges in quantum information science education: expert insights to guide framework development for interdisciplinary teaching and evolution of common language," *EPJ Quantum Technology*, vol. 13, no. 1, p. 2, Dec. 2025.

[21] W. B. Lane, M. Michelini, and C. Singh, "Editorial: Focused collection: Investigating and improving quantum education through research," *Physical Review Physics Education Research*, vol. 21, p. 010002, Apr. 2025.

[22] N. Skult *et al.*, "A Chronicle of Quantum Technologies in Game and Software Development," *IEEE Computer Graphics and Applications*, vol. 44, no. 5, pp. 14–26, 2024.

[23] L. Piispanen *et al.*, "Defining quantum games," *EPJ Quantum Technology*, vol. 12, no. 1, p. 7, Jan. 2025.

[24] N. Skult and J. Smed, "The Marriage of Quantum Computing and Interactive Storytelling," in *Games and Narrative: Theory and Practice*, B. Bostan, Ed. Cham: Springer International Publishing, 2022, pp. 191–206.

[25] S. N. Walck and N. C. Hansell, "Characterization and visualization of the state and entanglement of two spins," *European Journal of Physics*, vol. 22, no. 4, p. 343, Jul. 2001.

[26] G. F. Viamontes *et al.*, "Gate-level simulation of quantum circuits," in *Proceedings of the 2003 Asia and South Pacific Design Automation Conference*, ser. ASP-DAC '03. IEEE, 2003, pp. 295–301.

[27] B. Coecke and R. Duncan, "Interacting quantum observables," in *Proceedings of the 35th International Colloquium on Automata, Languages and Programming (ICALP), Part II*, L. Aceto *et al.*, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2008, pp. 298–310.

[28] H. Mäkelä and A. Messina, "*N* -qubit states as points on the Bloch sphere," *Physica Scripta*, vol. T140, p. 014054, Sep. 2010.

[29] M. Galambos and S. Imre, "New Method for Representation of Multi-qbit Systems Using Fractals," in *Proceedings of the International Conference on Quantum Nano and Micro Technologies (ICQNM)*. IARIA, Aug. 2011, pp. 52–56.

[30] M. Galambos and S. Imre, "Visualizing the Effects of Measurements and Logic Gates On Multi-Qubit Systems Using Fractal Representation," *International Journal On Advances in Systems and Measurements*, vol. 5, no. 1-2, pp. 1–10, 2012.

[31] G. DeepMind, "Gemini 2.5 flash and gemini 2.5 flash image model card," https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-2-5-Flash-Model-Card.pdf, 2025, accessed: 2025-11-24.

[32] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information: 10th Anniversary Edition*, 10th ed. USA: Cambridge University Press, 2011.

[33] I. G. Karafyllidis, "Visualization of the Quantum Fourier Transform Using a Quantum Computer Simulator," *Quantum Information Processing*, vol. 2, no. 4, pp. 271–288, Aug. 2003.

[34] J. B. Altepeter *et al.*, "Multiple-Qubit Quantum State Visualization," in *Proceedings of the Conference on Lasers and Electro-Optics/International Quantum Electronics Conference*. OSA, 2009, p. IWC1.

[35] C. Tahan, "Quantum mechanics is intuitive," APS March Meeting, San Antonio, Texas, Mar. 2015. [Online]. Available: https://tahan.com/charlie/talks/2015MMquantumintuitiontalk.pdf

[36] V. N. Chernega, O. V. Man'ko, and V. I. Man'ko, "Triangle Geometry of the Qubit State in the Probability Representation Expressed in Terms of the Triada of Malevich's Squares," *Journal of Russian Laser Research*, vol. 38, no. 2, pp. 141–149, Mar. 2017.

[37] A. Parakh, M. Subramaniam, and E. Ostler, "QuaSim: A virtual quantum cryptography educator," in *Proceedings of the 2017 IEEE International Conference on Electro Information Technology (EIT)*. IEEE, May 2017, pp. 600–605.

[38] Z. Tao *et al.*, "ShorVis: A Comprehensive Case Study of Quantum Computing Visualization," in *Proceedings of the 2017 International Conference on Virtual Reality and Visualization (ICVRV)*. IEEE, Oct. 2017, pp. 360–365.

[39] S. Lin, J. Hao, and L. Sun, "QuFlow: Visualizing Parameter Flow in Quantum Circuits for Understanding Quantum Computation," in *Proceedings of the 2018 IEEE Scientific Visualization Conference (SciVis)*. IEEE, Oct. 2018, pp. 37–41.

[40] J.-B. Lamy, "Dynamic Software Visualization of Quantum Algorithms with Rainbow Boxes," in *Proceedings of the 14th International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications*. SCITEPRESS - Science and Technology Publications, 2019, pp. 155–163.

[41] M. Miller and D. Miller, "GraphStateVis: Interactive Visual Analysis of Qubit Graph States and their Stabilizer Groups," in *Proceedings of the 2021 IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE, Oct. 2021, pp. 378–384.

[42] I. Arawjo *et al.*, "Notational Programming for Notebook Environments: A Case Study with Quantum Circuits," in *Proceedings of the 35th*

Annual ACM Symposium on User Interface Software and Technology. ACM, Oct. 2022, pp. 1–20.

[43] A. Jordon, A. Hawkins-Seagram, and U. Stege, "Implementation and Viusalization of Quantum Walks," in *Proceedings of the 2022 IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE, Sep. 2022, pp. 796–798.

[44] A. Suau *et al.*, "Vector Field Visualization of Single-Qubit State Tomography," in *Proceedings of the 2022 IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE, Sep. 2022, pp. 528–534.

[45] A. Suau, G. Staffelbach, and A. Todri-Sanial, "qprof: A gprof-Inspired Quantum Profiler," *ACM Transactions on Quantum Computing*, vol. 4, no. 1, pp. 1–28, Mar. 2023.

[46] A. Jordon *et al.*, "QWalkVis: Quantum Walks Visualization Application," in *Proceedings of the 2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE, Sep. 2023, pp. 87–93.

[47] Z. Wen *et al.*, "Quantivine: A Visualization Approach for Large-scale Quantum Circuit Representation and Analysis," *IEEE Transactions on Visualization and Computer Graphics*, pp. 1–11, 2023.

[48] P. Senapati *et al.*, "Advancing Comprehension of Quantum Application Outputs: A Visualization Technique," in *Proceedings of the 2023 International Workshop on Quantum Classical Cooperative*, ser. QCCC '23. New York, NY, USA: Association for Computing Machinery, 2023, pp. 25–28.

[49] S. Ruan *et al.*, "*VACSEN*: A Visualization Approach for Noise Awareness in Quantum Computing," *IEEE Transactions on Visualization and Computer Graphics*, vol. 29, no. 1, pp. 462–472, Jan. 2023.

[50] S. Ruan *et al.*, "*VENUS* : A Geometrical Representation for Quantum State Visualization," *Computer Graphics Forum*, vol. 42, no. 3, pp. 247–258, Jun. 2023.

[51] J. Bley *et al.*, "Visualizing entanglement in multiqubit systems," *Physical Review Research*, vol. 6, no. 2, p. 023077, Apr. 2024.

[52] Y.-H. Chou *et al.*, "Bridging the Quantum Education Gap: Hands-on Visualization Projects for Quantum Search Algorithm," in *Proceedings of the 2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE, Sep. 2024, pp. 112–121.

[53] M. Khan, P. J. Nair, and O. Di Matteo, "CircInspect: Integrating Visual Circuit Analysis, Abstraction, and Real-Time Development in Quantum Debugging," in *Proceedings of the 2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE, Sep. 2024, pp. 1000–1006.

[54] J. Chen *et al.*, "HammingVis: A visual analytics approach for understanding erroneous outcomes of quantum computing in hamming space," *Graphical Models*, vol. 136, p. 101237, Dec. 2024.

[55] H. Kim and K. N. Smith, "Interaction Techniques for User-Friendly Interfaces for Gate-Based Quantum Computing," in *Proceedings of the 2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE, Sep. 2024, pp. 482–483.

[56] S. Norrie *et al.*, "QGrover: Teaching Grover's Algorithm Through Visual Exploration," in *Proceedings of the 2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE, Sep. 2024, pp. 17–24.

[57] P. Debus, S. Issel, and K. Tscharke, "Quantum Machine Learning Playground," *IEEE Computer Graphics and Applications*, vol. 44, no. 5, pp. 40–53, Sep. 2024.

[58] S. Norrie *et al.*, "QNotation: A Visual Browser-Based Notation Translator for Learning Quantum Computing," in *Proceedings of the 2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE, Sep. 2024, pp. 25–33.

[59] S. Ruan *et al.*, "QuantumEyes: Towards Better Interpretability of Quantum Circuits," *IEEE Transactions on Visualization and Computer Graphics*, vol. 30, no. 9, pp. 6321–6333, Sep. 2024.

[60] D. Brahmbhatt *et al.*, "An open-source data storage and visualization platform for collaborative qubit control," *Scientific Reports*, vol. 14, no. 1, p. 22703, Sep. 2024.

[61] C. A. Steed *et al.*, "QVis: A Visual Analytics Tool for Exploring Noise and Errors in Quantum Computing Systems," in *Proceedings of the 2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE, Sep. 2023, pp. 211–214.

[62] J. Chae, C. A. Steed, and T. S. Humble, "Visual Analytics of Performance of Quantum Computing Systems and Circuit Optimization," in *Proceedings of the 2024 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*. IEEE, Jul. 2024, pp. 613–618.

[63] R. Seidel *et al.*, "Quantum backtracking in qrisp applied to sudoku problems," 2024. [Online]. Available: https://arxiv.org/abs/2402.10060

[64] S. Ruan *et al.*, "*VIOLET*: Visual Analytics for Explainable Quantum Neural Networks," *IEEE Transactions on Visualization and Computer Graphics*, vol. 30, no. 6, pp. 2862–2874, Jun. 2024.

[65] D. Huber and S. J. Glaser, "BEADS: a canonical visualization of quantum states for applications in quantum information processing," *New Journal of Physics*, vol. 27, no. 9, p. 094509, Sep. 2025.

[66] M. Karunathilaka *et al.*, "Intuit: Explain Quantum Computing Concepts via AR-based Analogy," in *Proceedings of the Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*, Apr. 2025, pp. 1–8.

[67] M. McGuffin and J.-M. Robert, "Visualizing quantum circuits: State vector difference highlighting and the half-matrix," *ACM Transactions on Quantum Computing*, vol. 7, no. 3, Jul. 2026. [Online]. Available: https://doi.org/10.1145/3786463

[68] S. J'oczik *et al.*, "Qcanvas: An Interactive Browser-Based Quantum Circuit Design and Simulation Platform," in *Proceedings of the 2025 IEEE International Conference on Quantum Software (QSW)*. IEEE, Jul. 2025, pp. 137–140.

[69] H. Patel, "Simulation and Visualization of Quantum Algorithms in Wolfram Mathematica: An Interactive Toolkit for Quantum Computing Education," *International Journal of Advanced Multidisciplinary Research and Studies*, vol. 5, no. 1, pp. 1204–1208, 2025.

[70] A. Heim *et al.*, "VisQI: A visual analysis tool for the debugging of quantum image processing algorithms," *Computer Graphics Forum*, p. e70651, 2026.

[71] Z. Wen *et al.*, "QuRAFT: Enhancing Quantum Algorithm Design by Visual Linking between Mathematical Concepts and Quantum Circuits," *IEEE Transactions on Visualization and Computer Graphics*, no. 01, pp. 1–15, Dec. 2025.

[72] F. Schinkel, "state-o-gram –A Novel 2D Visualization for Quantum States," Aug. 2025. [Online]. Available: http://arxiv.org/abs/2508.18390

[73] G. Sanderson, "But what is quantum computing? (Grover's Algorithm)," https://www.3blue1brown.com/lessons/grover, Apr. 2025, 3Blue1Brown. Accessed: 2026-01-23.

[74] T. Scruby, "Visualising quantum product codes," 2025. [Online]. Available: https://arxiv.org/abs/2507.11577

[75] P. Senapati *et al.*, "Visualization of Noisy and Less Noisy Computational Basis States in Quantum Computing," in *Proceedings of the 2025 IEEE International Conference on Quantum Software (QSW)*. IEEE, Jul. 2025, pp. 12–21.

[76] S. Ruan *et al.*, "Towards explainable quantum ai: Informing the encoder selection of quantum neural networks via visualization," *IEEE Transactions on Visualization and Computer Graphics*, vol. 32, no. 6, pp. 4840–4850, 2026.

[77] J. Johansson, P. Nation, and F. Nori, "Qutip: An open-source python framework for the dynamics of open quantum systems," *Computer Physics Communications*, vol. 183, no. 8, pp. 1760–1772, 2012.

[78] J. Johansson, P. Nation, and F. Nori, "Qutip 2: A python framework for the dynamics of open quantum systems," *Computer Physics Communications*, vol. 184, no. 4, pp. 1234–1240, 2013.

[79] N. Lambert *et al.*, "Qutip 5: The quantum toolbox in Python," *Physics Reports*, vol. 1153, pp. 1–62, 2025.

[80] R. S. Smith, M. J. Curtis, and W. J. Zeng, "A practical quantum instruction set architecture," 2016.

[81] Cirq Developers, "Cirq," Aug. 2025. [Online]. Available: https://doi.org/10.5281/zenodo.16867504

[82] D. K. Tuckett, "Tailoring surface codes: Improvements in quantum error correction with biased noise," 2020, (https://github.com/qecsim/qecsim).

[83] S. Sivarajah *et al.*, "TKET: A Retargetable Compiler for NISQ devices," *Quantum Science and Technology*, vol. 6, Nov. 2020.

[84] S. Efthymiou *et al.*, "Qibo: a framework for quantum simulation with hardware acceleration," *Quantum Science and Technology*, vol. 7, no. 1, p. 015018, Dec. 2021.

[85] C. Gidney, "Stim: a fast stabilizer circuit simulator," *Quantum*, vol. 5, p. 497, Jul. 2021.

[86] R. J. Hill *et al.*, "qBraid-SDK: Platform-agnostic quantum runtime framework." Jan. 2025. [Online]. Available: https://github.com/qBraid/qBraid

[87] K. Singhal *et al.*, "Q# as a quantum algorithmic language," *Electronic Proceedings in Theoretical Computer Science*, vol. 394, pp. 170–191, Nov. 2023.

[88] E. Huang *et al.*, "Tailoring three-dimensional topological codes for biased noise," *PRX Quantum*, vol. 4, p. 030338, Sep. 2023.

[89] P. Weinberg *et al.*, "Queracomputing/bloqade-python: v0.15.11," May 2024. [Online]. Available: https://doi.org/10.5281/zenodo.11114110

[90] D. Seitz *et al.*, "Qadence: A differentiable interface for digital and analog programs," *IEEE Software*, vol. 42, no. 6, pp. 44–54, 2025.

[91] R. Seidel *et al.*, "Qrisp: A framework for compilable high-level programming of gate-based quantum computers," 2024. [Online]. Available: https://arxiv.org/abs/2406.14792

[92] M. P. Harrigan *et al.*, "Expressing and Analyzing Quantum Algorithms with Qualtran," 2024. [Online]. Available: https://arxiv.org/abs/2409.04643

[93] Horizon Quantum Computing, "Triple alpha: Integrated development environment for quantum computing," https://www.horizonquantum.com/triple-alpha, 2025, accessed: 2026-01-22.

[94] P. Migdał *et al.*, "Visualizing quantum mechanics in an interactive simulation – Virtual Lab by Quantum Flytrap," *Optical Engineering*, vol. 61, no. 08, Jun. 2022.

[95] F. Chiarello and M. G. Castellano, "Board Games and Board Game Design as Learning Tools for Complex Scientific Concepts: Some Experiences," *International Journal of Game-Based Learning*, vol. 6, no. 2, pp. 1–14, Apr. 2016.

[96] D. Carberry *et al.*, "Board Games to teach Quantum Technologies to Engineers: A Peer Instruction Approach," *SSRN Electronic Journal*, 2022.

[97] A. Goff, "Quantum tic-tac-toe: A teaching metaphor for superposition in quantum mechanics," *American Journal of Physics*, vol. 74, no. 11, pp. 962–973, Nov. 2006.

[98] P. Kaye, R. Laflamme, and M. Mosca, *An introduction to quantum computing*, ser. Oxford scholarship online. Oxford: Oxford University Press, 2020.

[99] IBM Quantum, "IBM Quantum Composer," 2025, accessed: 2026-01-18. [Online]. Available: https://quantum.cloud.ibm.com/composer

[100] M. Schuld, R. Sweke, and J. J. Meyer, "Effect of data encoding on the expressive power of variational quantum-machine-learning models," *Physical Review A*, vol. 103, p. 032430, Mar. 2021.

[101] B. Coecke and A. Kissinger, *Picturing Quantum Processes: A First Course in Quantum Theory and Diagrammatic Reasoning*. Cambridge University Press, 2017.

[102] A. Kissinger and J. van de Wetering, *Picturing Quantum Software: An Introduction to the ZX-Calculus and Quantum Compilation*. Preprint, 2024.

[103] B. Coecke and S. Gogioso, *Quantum in Pictures: A New Way to Understand the Quantum World*, 1st ed. Cambridge Quantum, Feb. 2023.

[104] R. Wille, L. Burgholzer, and M. Artner, "Visualizing Decision Diagrams for Quantum Computing (Special Session Summary)," in *Proceedings of the 2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, Feb. 2021, pp. 768–773.

[105] R. Wille, S. Hillmich, and L. Burgholzer, "Decision Diagrams for Quantum Computing," in *Design Automation of Quantum Computers*, R. O. Topaloglu, Ed. Cham: Springer International Publishing, 2023, pp. 1–23.

[106] D. Limberger, W. Scheibel, J. Döllner, and M. Trapp, "Visual variables and configuration of software maps," *Journal of Visualization*, vol. 26, no. 1, pp. 249–274, Feb. 2023. [Online]. Available: https://doi.org/10.1007/s12650-022-00868-1

[107] S. P. Feyer *et al.*, "2D, 2.5D, or 3D? An exploratory study on multilayer network visualisations in virtual reality," *IEEE Transactions on Visualization and Computer Graphics*, vol. 30, no. 1, pp. 469–479, 2024.

[108] E. R. Tufte, *The visual display of quantitative information*. USA: Graphics Press, 1986.

[109] T. Munzner, *Visualization Analysis and Design*, ser. AK Peters Visualization Series. Boca Raton, FL: CRC Press, 2014.

[110] QuTube. Decoherence on a Bloch sphere. QuTech. [Online]. Available: https://www.qutube.nl/quantum-computer-12/decoherence-on-a-bloch-sphere

[111] G. Passante and A. Kohnle, "Interactive homework to support student learning of measurement uncertainty in quantum mechanics," *Physical Review Physics Education Research*, vol. 20, p. 020131, 2024.

[112] V. Borish and H. J. Lewandowski, "Affordances and challenges of incorporating a remote, cloud-accessible quantum experiment into undergraduate courses," *Physical Review Physics Education Research*, vol. 21, p. 010133, 2025.

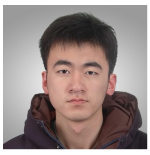
[113] S. Cao *et al.*, "Qcaleval: Benchmarking vision-language models for quantum calibration plot understanding," 2026. [Online]. Available: https://arxiv.org/abs/2604.25884

Feng Liang is a research engineer in the College of Computing and Data Science, Nanyang Technological University. His research interests focus on Visualization for Quantum Computing and AI for Science. He received his master's degree from KAUST in 2022 and his bachelor's degree from SUSTech in 2017. For more information, kindly visit https://fengliang.io/.

Xiaolin Wen is currently a Ph.D student in the College of Computing and Data Science, Nanyang Technological University (NTU). His research interests mainly focus on visualization for FinTech and LLM-assisted design study. He received his master's degree in Computer Science and Technology from Sichuan University in 2023 and his dual bachelor's degree in Computer Science and Financial Engineering from Sichuan University in 2016. For more information, kindly visit https://wenxiaolin.com/.

Shixian Zhou is currently working toward the MPhil degree with the School of Media and Design, Hangzhou Dianzi University, Hangzhou, China. He also studied as a visiting student at Nanyang Technological University, Singapore. His research interests include visual analytics, brain-computer interfaces, and online learning data analysis.

Shaolun Ruan is a postdoctoral researcher at the University of Konstanz. His research interests are Data Visualization and HCI. He focuses on addressing the explainability and usability problems in LLMs, quantum computing, and data storytelling. He received his Ph.D. and Bachelor's degree from Singapore Management University and the University of Electronic Science and Technology of China. Visit https://shaolun-ruan.com/ for more information.



Yuxuan Du is an Assistant Professor in the College of Computing and Data Science at Nanyang Technological University (NTU), Singapore. His research lies at the intersection of AI and quantum science, with a focus on quantum machine learning, quantum computing, and AI for Science. He is the author of one monograph and more than 50 publications in leading journals and conferences, including Nature Reviews Physics, Nature Communications, Physical Review Letters, ICML, and NeurIPS.

Qiang Guan is an Associate Professor in the Department of Computer Science at Kent State University. He is also a guest scientist at Los Alamos National Laboratory. His research interests include quantum computing, noise mitigation, quantum cloud and high-performance computing. He received his Ph.D. in Computer Science and Engineering from the University of North Texas in 2014, his M.S. in Information Engineering from Myongji University in 2008, and his B.S. in Communication Engineering from Northeastern University in 2005.

Jun Ye is a Principal Scientist at CETC International Cornerstone Quantum Industry (Suzhou) Co., Ltd. His research focuses on pulse-level quantum hardware emulation platforms, quantum dynamics simulations for open quantum systems, tensor-network and other high-performance simulation methods for quantum systems, as well as hybrid quantum-classical computing and its applications. He earned his B.Sc. from Central South University and his Ph.D. in Materials Science and Engineering from Nanyang Technological University, Singapore.

Yong Wang is currently an assistant professor in the College of Computing and Data Science, Nanyang Technological University. His research interests include data visualization, HCI and human-AI collaboration, with an emphasis on their application to FinTech, quantum computing and education. He obtained his Ph.D. in Computer Science from Hong Kong University of Science and Technology. For more details, please refer to http://yong-wang.org.